
Learning to Search, Searching to Learn: A Closed-Loop Framework for Large-Scale Vehicle Problems

Anonymous Author(s)

Affiliation

Address

email

Abstract

Large-scale Vehicle Routing Problems (VRPs) face two long-standing difficulties. On the one hand, many scalable methods rely on partitioning, local candidate restriction, or staged decision making to control computation, which weakens their modeling of global structure. On the other hand, although many methods introduce search at test time to improve the final solution, search is still typically used only as a one-shot post-processing step after model prediction. The model makes a prediction, search repairs it, and little sustained feedback is formed between the two. Improved structural states are rarely fed back to the model for subsequent inference, and high-quality search solutions are seldom turned into later training supervision. To address this issue, we propose **LSL (Learning to Search, Searching to Learn)**, a closed-loop learning-search framework for large-scale VRP. LSL first predicts search-friendly structural priors on a sparse candidate graph, and search then iteratively refines the current solution under the guidance of these priors. In turn, search does not leave the system after one round of refinement. At inference time, the structural states returned by search are fed back to the model for the next round of prediction, while at training time, multiple high-quality search solutions are reorganized into row-wise soft targets for model update. In this way, learning tells search where to explore, and search tells the model which structures are worth learning. Experiments show that LSL achieves strong scalability, efficiency, and solution quality across multiple large-scale VRP benchmarks.

1 Introduction

The Vehicle Routing Problem (VRP) is a central problem in combinatorial optimization, with broad applications in logistics, transportation, and supply chain management [1–4]. Because VRP is NP-hard, exact optimization quickly becomes impractical at large scale [5]. In practice, strong heuristics and metaheuristics such as LKH-style search and hybrid genetic search remain highly competitive, but they depend heavily on human expertise and hand-crafted algorithm design [6, 7]. Neural Combinatorial Optimization (NCO) offers a different route. Instead of manually designing every solving rule, it learns solving policies or structural priors directly from data [8–14]. While this idea has worked well on small and medium instances, making it work reliably on ultra-large VRP remains difficult.

One core difficulty is how a learning-based solver can preserve global-structure modeling under a controlled compute budget when the instance size grows to thousands, tens of thousands, or even one hundred thousand nodes. Recent work has improved cross-scale generalization through stronger constructive solvers, better inductive biases, or test-time adaptation [15–21]; another line scales to larger instances through decomposition, hierarchical solving, local candidate restriction, or staged

revision [22–28]. These directions have significantly advanced large-scale VRP, but they also reveal a common tension: to control complexity, solvers increasingly restrict decisions to local neighborhoods, local subproblems, or staged pipelines, while the long-range cross-region and cross-route couplings that often determine overall quality are more easily weakened in the process. This is especially critical in ultra-large VRP, because many local connections can only be judged properly in the context of a full route, or even the coordination among multiple routes.

A second core difficulty is that classical heuristic search, although powerful, is still usually treated as a one-shot post-processor in existing neural solvers. A large body of work shows that edge heatmaps, candidate-edge scores, or initial solutions produced by neural models can substantially improve final results when combined with downstream search or refinement [29–37]. Along this line, NeuroLKH further demonstrates the effectiveness of combining neural guidance with classical heuristic search: the learned module can provide valuable candidate information to an LKH-style backend and thereby improve final solution quality [38]. However, in most methods this combination is still one-way. The model predicts once and heuristic search refines once, but the structural information gradually revealed along the search trajectory is neither fed back to the model during the same inference run nor systematically converted into later training supervision. In other words, most existing neural-heuristic methods remain closer to an open-loop pipeline: heuristic search can improve the current instance, but it is not yet a true closed-loop component of learning. Although recent work has started to question the limitation of treating search only as post-hoc refinement [39–42], how to unify learning with a classical heuristic search backend into a closed-loop system that operates in both inference and training remains open for large-scale VRP.

In this paper, we propose **LSL (Learning to Search, Searching to Learn)**, a closed-loop framework that connects learning and search at both inference time and training time. *Learning to Search* means that the model no longer attempts to output a complete route in one shot. Instead, it predicts a sparse edge heatmap on a sparse candidate graph, and heuristic search uses this heatmap as guidance. *Searching to Learn* closes the loop in two ways. At inference time, LSL does not stop after one pass of learning followed by one pass of search. The model updates its heatmap using both the current instance and the current route structure returned by search, and search continues refining the incumbent under the updated heatmap, forming an inner learning-search loop. At training time, LSL collects multiple high-quality refined solutions, projects them onto the same candidate graph, and converts them into row-wise soft targets for later model updates. These targets place higher mass on edges repeatedly supported by good solutions, while keeping uncertainty where several local connections remain plausible. As a result, the model learns a heatmap that is more useful for downstream search, instead of being forced to commit too early to a single route. More importantly, this training-side loop also allows LSL to gradually generate and accumulate high-quality labeled data when large-scale supervision is scarce, for example by bootstrapping high-quality TSP100K training data within practical time. In this way, LSL lets learning and search work together on the current instance during inference, and also turns search outcomes into stronger supervision during training.

To make this loop practical on ultra-large instances, we instantiate LSL with three components, a *sparse diffusion transformer backbone*, an inference-time learning-search refinement loop, and a training-time search-feedback supervision loop. The backbone operates directly on sparse candidate edges and predicts the edge heatmap consumed by search, the refinement loop repeatedly uses the updated heatmap to improve the current solution under a fixed budget, and the training loop turns multiple high-quality refined solutions into supervision for later learning. This design also brings a concrete scaling advantage. When the number of candidate edges is fixed at $O(nK)$, the dominant memory cost and computation of the learning module both grow approximately linearly with problem size, instead of quadratically as in dense $O(n^2)$ graphs. This allows LSL to handle ultra-large instances on a single consumer-grade GPU without relying on divide-and-conquer or hard decomposition, while still giving search heatmaps that retain global structural information.

Experiments on large-scale TSP and CVRP benchmarks show that LSL achieves an excellent balance among solution quality, inference efficiency, and scalability, reaching state-of-the-art or highly competitive results in multiple settings. More importantly, these results show that the closed loop itself is valuable: the same classical heuristic search process can continually improve the current solution during inference and continually generate stronger supervision during training. Overall, our contributions are fourfold.

- 92 • We introduce **LSL**, a closed-loop framework that unifies neural prior learning with a classical
93 heuristic search backend for large-scale VRP, rather than treating heuristic search only as a
94 one-shot post-hoc refinement module.
- 95 • We design a practical ultra-large-scale instantiation centered on a sparse diffusion trans-
96 former backbone over sparse candidate graphs, achieving near-linear scalability with respect
97 to problem size while retaining global structural information as much as possible.
- 98 • We propose row-wise multi-solution soft supervision induced by heuristic-search trajectories
99 and show on large-scale TSP/CVRP benchmarks that this design improves scalability,
100 efficiency, and solution quality.
- 101 • We further show that LSL can bootstrap high-quality 100K-scale training data that can serve
102 as reusable supervision resources for subsequent learning-based routing research.

103 2 Closed-loop learning-search framework

104 2.1 VRP formulation

105 VRP can be uniformly formulated as a combinatorial optimization problem on a global graph. Given
106 an instance, let its global graph be $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the node set and $\mathcal{E} = \{(i, j) \mid i, j \in$
107 $\mathcal{V}, i \neq j\}$ is the global edge set. Let $\mathcal{X}(\mathcal{G})$ denote the feasible solution space of instance $\mathcal{G}$, and
108 let $\mathbf{x} \in \mathcal{X}(\mathcal{G})$ be a feasible solution defined on the global graph. Uniformly, $\mathbf{x}$ can be viewed as
109 an encoding of structural selection relations over the global edge domain. The unified optimization
110 objective of VRP can therefore be written as

$$\mathbf{x}^*(\mathcal{G}) = \arg \min_{\mathbf{x} \in \mathcal{X}(\mathcal{G})} C(\mathbf{x}; \mathcal{G}), \quad (1)$$

111 where $C(\mathbf{x}; \mathcal{G})$ denotes the total cost of solution $\mathbf{x}$. Under this formulation, TSP and CVRP differ
112 mainly in the constraint structure of $\mathcal{X}(\mathcal{G})$. For TSP, feasible solutions must satisfy the single-tour
113 constraint and the unique-visit constraint. For CVRP, in addition to the unique-service constraint on
114 customers, route-depot connectivity and vehicle-capacity constraints must also be satisfied. In the
115 remainder of the paper, we no longer model $\mathbf{x}$ directly on the global edge domain, but instead map it
116 to a structural representation that is better suited to large-scale learning and search. Before that, we
117 first briefly review existing diffusion-based offline solution-distribution learning methods.

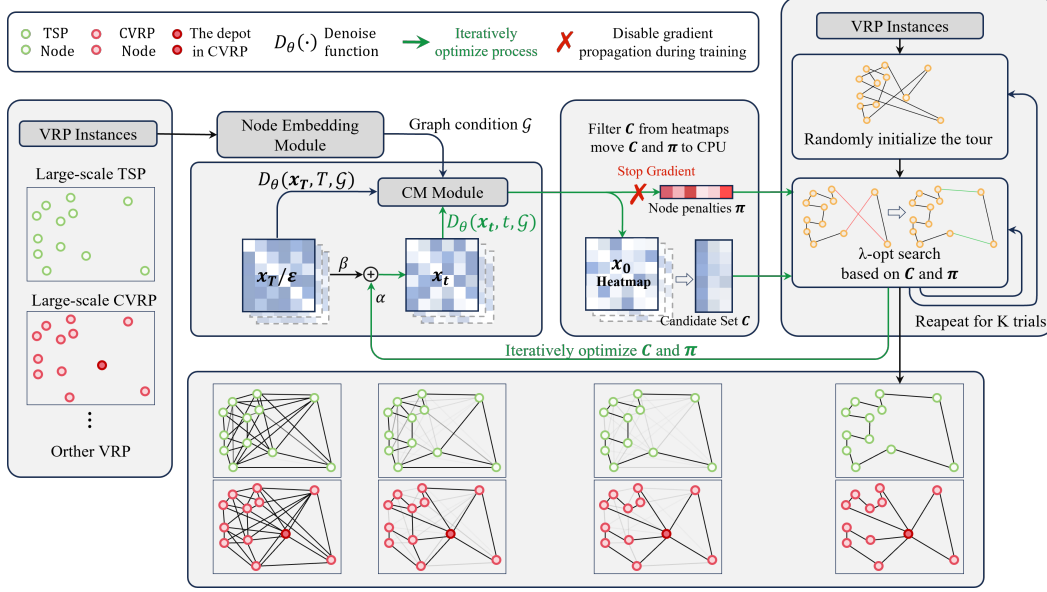


Figure 1: **Overview of LSL.** Starting from a large-scale VRP instance (left), a node embedding module constructs sparse instance features on the candidate graph. Conditioned on the instance, the timestep, and the current structural state returned by the previous search round, the sparse diffusion denoiser D_θ predicts an updated structural prior on the same sparse support (middle left). The CM module injects incumbent-solution context, so prediction depends on both static geometry and the current structure revealed by search. The predicted heatmap is filtered into a sparse candidate set C and node penalties π (middle right); the “Stop Gradient” marker indicates that this discrete extraction and the downstream heuristic search are used as non-differentiable feedback interfaces. On the right, a concrete Guided-LKH-style backend initializes and refines tours or routes by λ -opt search under the guidance of C and π , retaining the best results across multiple trials. The green arrows show the inference-time closed loop: improved solutions are projected back to the sparse graph as the next structural state, allowing learning and search to iteratively co-refine the candidate support. During training, multiple high-quality search outputs are further projected onto the candidate graph to form row-wise soft targets. The bottom TSP/CVRP examples illustrate how the sparse support becomes cleaner while preserving globally meaningful connections.

118 2.2 Diffusion-based offline solution-distribution learning

119 Recent generative neural combinatorial optimization methods no longer treat solving as a one-step
 120 mapping from an instance to a final solution. Instead, they first learn an instance-conditioned
 121 distribution of high-quality solutions offline. Given a high-quality reference solution $\mathbf{x}^*$ and a feasible
 122 suboptimal solution $\mathbf{x}^{\text{sub}}$, one can define an interpolation trajectory

$$\mathbf{x}_t = t\mathbf{x}^{\text{sub}} + (1-t)\mathbf{x}^*, \quad t \in [0, 1]. \quad (2)$$

123 In this setting, the drift field $\mathbf{v}$ is chosen to follow the linear direction from $\mathbf{x}^*$ to $\mathbf{x}^{\text{sub}}$, i.e.,

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, t, G) = \mathbf{x}^{\text{sub}} - \mathbf{x}^*. \quad (3)$$

124 Because $\mathbf{x}^*$ is unknown when simulating this ODE flow, the velocity field is parameterized by a
 125 neural network and trained with

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}^{\text{sub}}, G} \left[\int_0^1 \|\mathbf{v}_\theta(\mathbf{x}_t, t, G) - (\mathbf{x}^{\text{sub}} - \mathbf{x}^*)\|_2^2 dt \right]. \quad (4)$$

126 2.3 Search-friendly sparse diffusion transformer

Existing diffusion- or flow-matching-based combinatorial optimization methods differ in state spaces and parameterizations, but most still learn around a single high-quality reference solution $\mathbf{x}^*$. Even when search is introduced at test time, it mainly optimizes candidate solutions inside the model’s learned solution space or generation trajectory. For large-scale VRP, this creates a key limitation: the model learns how to approximate one particular high-quality solution, rather than how to provide genuinely useful structural guidance for downstream discrete search.

Moreover, because states in $\mathbf{x}$ -space are usually interpreted as feasible solutions themselves, or at least as intermediate variables moving toward feasibility, the generation process is naturally pushed toward combinatorial constraints. This is especially restrictive for problems such as CVRP, where capacity constraints, depot-connectivity constraints, and route-structure constraints all coexist. Such a design may be useful for directly recovering one high-quality tour or route set, but it is not necessarily the right object for downstream search. For search, what is often more valuable is not an object already heavily constrained toward feasibility and prematurely committed to local decisions, but a prior distribution that expresses structural preference without being forced to satisfy every final combinatorial constraint. In other words, search prefers a signal about where it is worth searching, rather than a half-finished solution for which the model has already done most of the search.

For this reason, we no longer treat the time-conditioned state merely as a perturbed solution variable, but as a time-conditioned structural prior defined on the graph. Specifically, we use $\mathcal{H}$ rather than $\mathbf{x}$ to denote the structural state processed by the model, where $\mathcal{H}_t$ is the structural prior at timestep t . When t is small, $\mathcal{H}_t$ is closer to high-quality structure; when t is large, $\mathcal{H}_t$ retains weaker, blurrier, or more incomplete structural information. In our setting, the role of the time variable t is therefore not only to control noise strength, but also to control how informative the current structural prior is about high-quality solution regions. Crucially, $\mathcal{H}$ itself is not required to represent a feasible solution under the combinatorial constraints. It is a pure structural distribution on graph edges and therefore leaves more room for downstream search to adjust.

To make this structural-prior learning practical on ultra-large instances, we further map the global graph $\mathcal{G}$ to a sparse candidate graph

$$\mathcal{G}_K = (\mathcal{V}, \mathcal{E}_K), \quad \mathcal{E}_K = \{(i, j) \mid j \in \mathcal{N}_K(i), |\mathcal{N}_K(i)| = K\}, \quad (5)$$

where $\mathcal{N}_K(i)$ denotes the K retained candidate neighbors of node i . All later structural states, model predictions, and supervision targets are defined on this sparse edge domain, which compresses the dominant computation from $O(N^2)$ on the dense graph to $O(NK)$.

On this basis, we construct a sparse diffusion Transformer as the learning module. Given the current instance graph $\mathcal{G}_K$, timestep t , and current structural state $\mathcal{H}_t$, the model learns the mapping $D_\theta(\mathcal{G}_K, \mathcal{H}_t, t)$ and outputs a row-wise structural distribution on the same sparse candidate edge domain, namely $\mathcal{H}_{\text{guide}} = D_\theta(\mathcal{G}_K, \mathcal{H}_t, t)$. Here $\mathcal{H}_{\text{guide}}$ is not interpreted as the final solution itself, but as a structural guidance signal consumed by downstream search. In other words, the purpose of this module is not to recover one high-quality route directly, but to output a prior over which local connections deserve more attention from the subsequent search procedure.

Overall, the role of this sparse diffusion Transformer is not to recover one single high-quality solution from an intermediate state, but to learn a structural prior that explicitly serves downstream heuristic search. The sparse online softmax is one of the practical ingredients that makes this search-friendly sparse learner viable at scale.

Architecture At the architectural level, the learning module consists of a sparse encoder and a time-conditioned sparse decoder. The encoder propagates node geometry and static instance information over the candidate edge domain of $\mathcal{G}_K$, while the decoder injects both the timestep t and the current

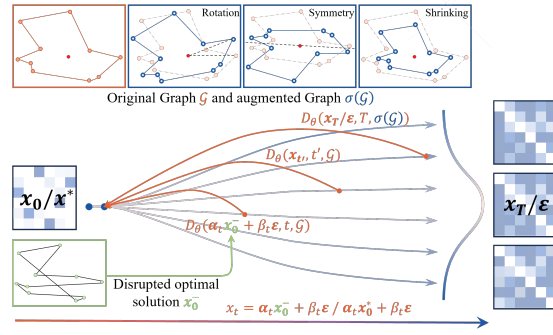


Figure 2: **Structural-state construction at inference time.** A sparse structural heatmap is perturbed and then interpolated to produce intermediate states $\mathcal{H}_t$ for denoising.

181 structural state $\mathcal{H}_t$ into the same sparse structural domain. Concretely, at decoder layer ℓ and attention
 182 head h , the attention score from node i to candidate node $j \in \mathcal{N}_K(i)$ is

$$e_{ij}^{(\ell,h)} = \frac{\langle \mathbf{q}_i^{(\ell,h)}, \mathbf{k}_j^{(\ell,h)} \rangle}{\sqrt{d_h}} + b_{ij}^{(\ell)}(\mathcal{H}_t, t), \quad (6)$$

183 where $\mathbf{q}_i^{(\ell,h)}$ and $\mathbf{k}_j^{(\ell,h)}$ denote the query and key of head h , d_h is the hidden dimension of one head,
 184 and $b_{ij}^{(\ell)}(\mathcal{H}_t, t)$ is the additive bias jointly induced by the current structural state and the timestep. In
 185 this way, the model not only perceives the geometric and topological context of the current instance,
 186 but also explicitly uses information about which structural patterns have already emerged in the
 187 current solution state.

188 To maintain numerical stability and efficiency at large K , we do not compute a dense row-wise
 189 softmax explicitly over all candidates. Instead, we use blockwise sparse online softmax. Specifically,
 190 we partition the candidate set $\mathcal{N}_K(i)$ into blocks $\{\mathcal{B}_b(i)\}_b$, where $\mathcal{B}_b(i) \subseteq \mathcal{N}_K(i)$ denotes the b -th
 191 candidate block. Let $m_i^{(\ell,h,b)}$ and $z_i^{(\ell,h,b)}$ be the running row maximum and running partition sum
 192 after processing block b . Then

$$m_i^{(\ell,h,b)} = \max\left(m_i^{(\ell,h,b-1)}, \max_{j \in \mathcal{B}_b(i)} e_{ij}^{(\ell,h)}\right), \quad (7)$$

$$z_i^{(\ell,h,b)} = z_i^{(\ell,h,b-1)} \exp\left(m_i^{(\ell,h,b-1)} - m_i^{(\ell,h,b)}\right) + \sum_{j \in \mathcal{B}_b(i)} \exp\left(e_{ij}^{(\ell,h)} - m_i^{(\ell,h,b)}\right). \quad (8)$$

194 This form allows us to perform stable normalization directly on the sparse candidate domain without
 195 materializing a dense softmax over the whole row. For our setting, this is not merely an implemen-
 196 tation optimization, but a key ingredient that allows the sparse diffusion Transformer to preserve
 197 $O(NK)$ complexity on ultra-large instances while maintaining training stability and inference effi-
 198 ciency.

199 At the end of the decoder, we use a lightweight edge-scoring head to generate the final structural
 200 distribution on candidate edges. Let $\mathbf{g}_i$ denote the final-layer representation of node i . Then

$$s_{ij} = \langle \mathbf{g}_i, \mathbf{g}_j \rangle, \quad \mathcal{H}_{\text{guide},ij} = \frac{\exp(s_{ij})}{\sum_{j' \in \mathcal{N}_K(i)} \exp(s_{ij'})}, \quad j \in \mathcal{N}_K(i). \quad (9)$$

201 Figure 1 summarizes the full LSL pipeline. The illustration uses a concrete Guided-LKH-style
 202 instantiation to show how the predicted sparse heatmap is converted into discrete search guidance; in
 203 the equations below, we keep the backend abstract as $\mathcal{S}$ so that the same closed-loop interface also
 204 covers other sparse search realizations.

205 2.4 Inference-time learning-search refinement

206 At inference time, LSL runs multiple learning-search cycles on the same instance. Let $\mathcal{R}^{(c)}$ be the
 207 incumbent route set at cycle c , and let $\mathcal{H}^{(c)} = \text{Proj}(\mathcal{R}^{(c)}; \mathcal{G}_K)$ be its projected structural heatmap.
 208 In TSP, this route set degenerates to one tour; in CVRP, it contains multiple routes from the same
 209 feasible solution. Conditioned on this structural state, the denoiser runs a full inference pass and
 210 outputs a guidance heatmap $\mathcal{H}_{\text{guide}}^{(c)}$, and search then spends budget B to refine the incumbent route
 211 set under this guidance:

$$\begin{aligned} \mathcal{H}_{\text{guide}}^{(c)} &= D_\theta(\mathcal{G}_K, \mathcal{H}^{(c)}), \\ \mathcal{R}^{(c+1)} &= \mathcal{S}(\mathcal{R}^{(c)}, \mathcal{H}_{\text{guide}}^{(c)}, B), \\ \mathcal{H}^{(c+1)} &= \text{Proj}(\mathcal{R}^{(c+1)}; \mathcal{G}_K). \end{aligned} \quad (10)$$

212 Here $\mathcal{H}_{\text{guide}}^{(c)}$ denotes the guidance heatmap produced by the sparse diffusion denoiser parameterized
 213 by θ and conditioned on the instance graph $\mathcal{G}_K$ and the current structural heatmap $\mathcal{H}^{(c)}$. We write
 214 $D_\theta(\mathcal{G}_K, \mathcal{H}^{(c)})$ for this full denoising call and suppress its internal time integration for readability.
 215 The scalar B is the search budget allocated to one learning-search cycle. In our implementation, $\mathcal{S}$

can be instantiated by heatmap-guided LKH refinement or sparse merge-style construction, but the interface itself is backend-agnostic.

This formulation makes each inference cycle explicit. The current route set is first projected into a structural state on the sparse candidate graph, learning converts that state into the next guidance heatmap, and search returns the updated route set under this guidance. The model therefore does not restart from the raw instance alone at every cycle. It also conditions on the latest structure that search has already uncovered.

Search, in turn, is no longer a one-shot repair module after prediction. It becomes an operator inside the inference loop, because the latest route set is immediately projected back to the same sparse graph and fed into the next model call. In practice, we optionally perturb the incumbent route set between consecutive cycles with a small random local move before the next prediction. This prevents the loop from repeatedly refining the same basin and encourages exploration of nearby but structurally different solution states, while keeping search and learning coupled through the same structural heatmap interface.

2.5 Training-time search feedback

During training, search no longer serves to further refine the current solution, but instead feeds stronger supervision back into learning. For each training instance, we retain multiple high-quality refined route sets $\{\mathcal{R}^{(m)}\}_{m=1}^M$ encountered along the local-search trajectory and project them onto the same sparse candidate graph to form an unnormalized search-consensus target

$$\bar{\mathcal{H}}_{ij} = \sum_{m=1}^M w_m \text{Proj}(\mathcal{R}^{(m)}; \mathcal{G}_K)_{ij}, \quad w_m \geq 0, \quad (11)$$

where w_m is the nonnegative weight assigned to route set $\mathcal{R}^{(m)}$, with better solutions receiving larger weights. We then normalize each row to obtain the final soft target

$$\mathcal{H}_{ij}^* = \frac{\bar{\mathcal{H}}_{ij}}{\sum_{j' \in \mathcal{N}_K(i)} \bar{\mathcal{H}}_{ij'}}. \quad (12)$$

Unlike a single hard route label, $\mathcal{H}^* \in \mathbb{R}_+^{N \times K}$ distills structural consensus from the search trajectory rather than from one fixed teacher solution. If multiple good solutions repeatedly support the same local connection, its probability mass becomes large. If several alternatives remain plausible, the row stays soft.

The second characteristic of the training loop is that the model is not trained only on clean teacher states. Instead, we construct a current structural heatmap $\mathcal{H}$ and a teacher-side structural heatmap $\hat{\mathcal{H}}$ on the same sparse graph, and interpolate them with the timestep:

$$\mathcal{H}_t = (1 - t)\mathcal{H} + t\hat{\mathcal{H}}. \quad (13)$$

Here $\mathcal{H}$ is obtained by projecting a current route set, while $\hat{\mathcal{H}}$ is obtained by projecting either a high-quality teacher solution or its disrupted version. The point of this construction is not to redefine the model, but to train it on structured yet imperfect intermediate states that more closely match what search actually feeds back during inference.

The training objective is a row-wise sparse cross-entropy between the predicted heatmap and the search-induced soft target,

$$\mathcal{L}_{\text{LSL}} = -\frac{1}{N} \sum_{i=1}^N \sum_{j \in \mathcal{N}_K(i)} \mathcal{H}_{ij}^* \log D_\theta(\mathcal{G}_K, \mathcal{H}_t, t)_{ij}. \quad (14)$$

Here $D_\theta(\mathcal{G}_K, \mathcal{H}_t, t)_{ij}$ is the probability assigned by the denoiser to candidate edge (i, j) in row i , conditioned on the sparse instance graph $\mathcal{G}_K$. For TSP, this is the full objective. For CVRP, the same formulation is retained, while depot-aware state construction and depot-sensitive reweighting are introduced in Appendix B.2.

The third characteristic of the training loop is that once search finds better solutions, we can directly convert them into new soft targets and merge them into the next round of training data:

$$\mathcal{D}^{(r+1)} = \mathcal{D}^{(r)} \cup \left\{ (\mathcal{G}_K, \mathcal{H}^{*,(r)}(\mathcal{G}_K)) \mid \mathcal{G}_K \in \mathcal{B}^{(r)} \right\}. \quad (15)$$

Table 1: Results on synthetic TSP and CVRP instances. Results marked with an asterisk (*) are from the original paper. The best results are bold. OOM: The method triggered an out-of-memory condition. The reported inference time is the total time required to solve the entire test set.

Method	TSP100			TSP1K			TSP10K		
	Length	Gap	Time	Length	Gap	Time	Length	Gap	Time
LKH3	7.76325	0.000%	56m	23.119	0.000%	8.2h	71.778	0.000%	8.8h
Concorde	7.76324	0.000%	34m	23.118	-0.003%	7.8h	72	0.310%	22.4h
Att-GCN+MCTS*	7.764	0.037%	15m	23.863	3.224%	13m	74.93	4.390%	1.8h
DIFUSCO*	7.78	0.240%	-	23.56	1.900%	12.1m	73.62	2.580%	47m
T2T*	7.76	0.060%	-	23.3	0.780%	54.7m	-	-	-
Fast-T2T*	7.76	0.010%	-	23.22	0.420%	18.3m	72.94	1.630%	-
GenSCO*	7.76	0.000%	-	23.13	0.036%	3.9m	-	-	-
H-TSP	-	-	-	24.718	6.912%	34s	77.75	8.320%	25m
GLOP	7.767	0.048%	1.4h	23.84	3.119%	2.4m	75.04	4.545%	25m
UDC- $\mathbf{x}_{50}$ ($\alpha = 50$)	7.788	0.319%	1.38h	23.53	1.782%	6.21m	OOM	OOM	OOM
POMO aug $\times 8$	7.774	0.138%	0.7m	32.5	40.577%	6.1m	OOM	OOM	OOM
ELG	7.781	0.229%	2.2m	25.738	11.328%	1.3m	OOM	OOM	OOM
BQ bs16	7.764	0.010%	20m	23.432	1.354%	21m	OOM	OOM	OOM
LEHD RRC1000	7.763	0.002%	2.3h	23.288	0.731%	5.2h	80.9	12.709%	3.6h
L2C-Insert Greedy	7.799	0.459%	0.86m	24.218	4.754%	0.35m	77.343	7.754%	1.06m
L2C-Insert ($I=1000$)	7.763	0.000%	8.0h	23.230	0.481%	46.4m	73.270	2.079%	16.64m
LSL ($C=1, B=128$)	7.763	0.000%	4s	23.119	0.000%	1.7m	71.800	0.030%	1.2m
LSL ($C=4, B=512$)	-	-	-	23.118	-0.002%	15.5m	71.775	-0.005%	16.9m
Method	CVRP100			CVRP1K			CVRP10K		
	Length	Gap	Time	Length	Gap	Time	Length	Gap	Time
HGS	15.56	0.000%	4.5h	41.161	0.000%	5h	226.177	0.000%	8h
LKH3	15.65	0.540%	12h	42.156	2.416%	6.1h	290.575	28.472%	10.3h
GLOP-G (LKH3)	-	-	-	45.753	11.154%	1.93m	276.252	22.140%	27.98m
UDC- $\mathbf{x}_{50}$ ($\alpha = 50$)	16.291	4.699%	2.69h	43.583	5.884%	11.96m	OOM	OOM	OOM
POMO aug $\times 8$	15.75	1.230%	0.86m	100.990	>100%	5.83m	OOM	OOM	OOM
ELG	15.84	1.760%	3.4m	46.847	13.812%	3.16m	277.685	22.773%	11.92m
BQ bs16	15.81	1.560%	18m	43.121	4.762%	15m	275.494	21.805%	1.36h
LEHD RRC1000	15.63	0.420%	2.0h	42.408	3.028%	4.15h	248.604	9.916%	8.1h
L2C-Insert Greedy	16.169	3.892%	1.4m	44.458	8.009%	1.17m	255.356	12.901%	2.88m
L2C-Insert ($I=1000$)	15.627	0.413%	10.6h	43.152	4.836%	2.05h	245.732	8.646%	34.72m
LSL ($C=1, B=128$)									
LSL ($C=4, B=512$)									

Here $\mathcal{D}^{(r)}$ is the training set after round r , $\mathcal{B}^{(r)}$ is the set of instance graphs revisited in that round, and $\mathcal{H}^{*,(r)}(\mathcal{G}_K)$ is the soft target mined from the latest search-refined solutions of instance $\mathcal{G}_K$. This gives LSL a simple self-bootstrapping mechanism at large scale. In particular, when high-quality labels are scarce for very large instances, the framework can keep rewriting its soft targets with search-refined solutions and gradually accumulate stronger supervision.

3 Experiments

In this section, we systematically evaluate LSL from four perspectives on large-scale TSP and CVRP. First, we compare it with classical solvers, constructive NCO methods, heatmap-based NCO methods, and two-stage large-scale routing methods on synthetic benchmarks to assess overall solution quality, inference efficiency, and scalability. Second, we evaluate cross-distribution generalization on real-world TSPLIB, VLSI, Art-TSP, and CVRPLIB instances. Third, we conduct ablations on the closed-loop mechanism and search-feedback supervision to verify whether the core design truly brings performance gains. Finally, we analyze the behavior of the framework under different inference budgets, cycle counts, candidate widths, and large-scale scaling curves.

3.1 Experimental setup

Datasets To systematically evaluate LSL across different problem scales, we build multi-scale synthetic datasets for both TSP and CVRP rather than training and testing at only a single scale.

Specifically, for TSP we generate four datasets, TSP100, TSP1K, TSP10K, and TSP100K, with 1.28M, 64K, 6.4K, and 640 instances, respectively. The 10K data are processed with LSL self-refinement, and the 100K data are bootstrapped by LSL. For CVRP we generate three datasets, CVRP100, CVRP1K, and CVRP10K, with 1.28M, 64K, and 6.4K instances, respectively; CVRP1K and CVRP10K are bootstrapped by LSL.

For real-world evaluation, we further use TSPLIB [43], VLSI, Art-TSP, CVRPLIB Set-X, and CVRPLIB Set-SL [44] to examine generalization under real and cross-distribution settings. In the main text we report aggregated results, while finer-grained per-instance results are deferred to the appendix.

Baselines We compare LSL with the following methods: **(1) Classical solvers:** Concorde [45], LKH3 [6], and HGS [46]; **(2) Constructive NCO methods:** POMO [12], BQ [14], LEHD [13], ELG [27], INVt [28], and L2C-Insert [21]; **(3) Heatmap-based or generative NCO methods:** AttGCN+MCTS [30], DIFUSCO [32], T2T [33], Fast-T2T [34], and GenSCO [35]; and **(4) Two-stage or large-scale methods:** H-TSP [22], GLOP [25], and UDC [26]. For baselines whose results are available only from the original papers and are difficult to reproduce fully under a unified environment, we directly cite the reported numbers and mark them with an asterisk in the tables.

Table 2: Results on synthetic TSP instances with 50K/100K nodes. The best results are bold.

Method	TSP50K			TSP100K		
	Length	Gap	Time	Length	Gap	Time
LKH3	159.93	0.000%	160h	225.99	0.000%	400h
GLOP	168.09	5.102%	24m	237.61	5.142%	1.04h
INVt greedy	171.42	7.184%	20.8h	242.26	7.199%	80h
LEHD RRC1000	OOM	OOM	OOM	OOM	OOM	OOM
L2C-Insert Greedy	171.778	7.408%	5.28m	242.757	7.419%	10.53m
L2C-Insert ($I=1000$)	166.063	3.835%	20.83m	237.109	4.920%	26.07m
LSL ($C=1, B=128$)						
LSL ($C=4, B=512$)						
LSL ($C=8, B=1024$)						

Inference protocol and metrics Figure 2 illustrates how the intermediate structural heatmaps fed to the denoiser are constructed.

At inference time, LSL first predicts structural priors on a sparse candidate graph, after which a heuristic backend performs discrete refinement under a fixed budget. Unless otherwise specified, we use $\text{aug} \times 8$ by default and report two representative configurations: a lightweight setting LSL ($C=1, B=128$) and a stronger setting LSL ($C=4, B=512$), where C denotes the number of learning-search cycles and B denotes the search budget per cycle. We report three metrics: **(1) average tour length / total cost (Length)**, **(2) the performance gap relative to a reference solver (Gap)**, and **(3) the total wall-clock time required to solve the full test set (Time)**. For runtime numbers taken from original papers, we treat them mainly as references rather than making overly strong absolute speed comparisons against times measured locally on unified hardware; finer-grained per-instance inference times are provided in the appendix.

3.2 Main results on synthetic benchmarks

Tables 1 and 2 report the main results on synthetic TSP and CVRP benchmarks. Overall, LSL achieves stable and competitive performance across scales. For TSP, LSL already reaches results close to strong classical heuristics on TSP100 and TSP1K; when the scale increases to 10K, 50K, and 100K, it still maintains extremely low gaps, whereas many constructive or dense heatmap-based methods degrade substantially or run out of memory. Especially at large scale, LSL consistently offers a better quality-time tradeoff than representative neural baselines and two-stage methods, suggesting that the combination of sparse structural priors and heuristic refinement is better suited to ultra-large instances.

For CVRP, LSL likewise shows good scalability. On CVRP100, CVRP1K, and CVRP10K, LSL consistently outperforms most neural baselines and achieves a better balance between solution quality and inference time. Compared with methods that construct routes directly, LSL does not require the model to resolve all discrete feasibility decisions by itself; instead, it combines depot-aware sparse structural guidance with subsequent heuristic refinement, which leads to more stable behavior as the problem scale grows.

Table 3: Comparison results on TSPLIB, VLSI, Art-TSP, and CVRPLIB instances.

Method	TSPLIB		VLSI		Art-TSP		CVRPLIB	
	≤ 1000	> 1000	≤ 1000	> 1000	≤ 1000	> 1000	≤ 1000	> 1000
UDC- $\mathbf{x}_{250}$ ($\alpha = 50$)	3.072%	-	-	-	-	-	9.033%	-
GLOP	1.153%	-	-	-	-	-	-	-
ELG	3.111%	-	-	-	-	-	7.039%	-
DIFUSCO	1.736%	-	-	-	-	-	-	-
Fast T2T	0.542%	-	-	-	-	-	-	-
GenSCO	0.184%	-	-	-	-	-	-	-
BQ bs16	1.557%	-	-	-	-	-	8.078%	-
L2C-Insert ($I=1000$)	0.308%	2.915%	-	-	-	-	3.201%	-
LSL ($C=1, B=128$)	0.003%		-		-		-	
LSL ($C=4, B=512$)								

Table 4: Ablation on the closed-loop mechanism at inference time. Different learning–search combinations are compared under the same or similar total budget.

Method	Learned guide	Guide refreshed	Search	TSP1K			TSP10K		
				Length	Gap	Time	Length	Gap	Time
Search-only (No guide)			✓						
One-shot Guide + single-solution hard-label guide	✓		✓						
One-shot Guide + multi-solution soft-target guide	✓		✓						
LSL Full ($C=4$)	✓	✓	✓						

3.3 Results on real-world benchmarks

Table 3 further summarizes results on real-world TSPLIB, VLSI, Art-TSP, and CVRPLIB benchmarks. Although these instances differ in distribution from the synthetic training data, LSL still shows strong generalization ability. Per-instance results are deferred to the appendix.

3.4 Is the learning–search closed loop really effective?

One core claim of LSL is that heuristic search should not be treated merely as a one-shot post-processing step after prediction, but as an explicit component inside the inference loop. To verify this, we compare four settings under the same search backend and identical or comparable budgets: **(1) Search-only**, which runs heuristic search without a learned prior; **(2) One-shot Guide + single-solution hard labels**, which predicts the structural prior once at the beginning, fixes it during search, and uses a single high-quality solution as one-hot supervision; **(3) One-shot Guide + multi-solution soft targets**, which also predicts the prior only once but uses multiple high-quality solutions to construct row-wise soft targets; and **(4) LSL Full**, the closed-loop variant proposed in this paper, which refreshes the guide from the current structural state at every cycle and feeds it back to the searcher.

Table 4 is used to quantify the differences among these inference paradigms. Under a fixed total budget, LSL Full is expected to consistently outperform one-shot or static variants. This would directly show that the gain does not come merely from running more rounds of search, but from the explicit bidirectional coupling of learning and search on the same instance. To further visualize this advantage, one may also report an anytime curve in the main text or appendix, showing how the gap changes with wall-clock time or cycle index.

3.5 More ablation studies

Beyond the closed-loop mechanism and the supervision form, we further analyze several key design choices in LSL.

Can LSL really help heuristic solvers escape local optima? The key question is not only the final endpoint, but whether, under the same or a similar budget, the learned guide can steer the search trajectory into a better basin. Two forms of evidence are particularly informative: the average improvement of LSL-guided search over search-only under a fixed budget, and, on manually constructed datasets with distant local-optimum traps, the average time required for search-only and closed-loop variants to escape those traps.

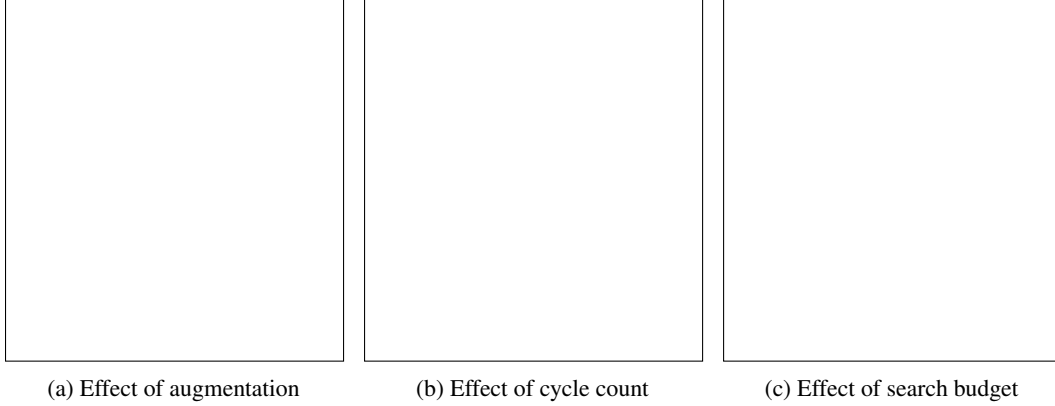


Figure 3: Effects of augmentation, cycle count, and search budget on the performance of LSL.

Effects of augmentation, cycle count, and search budget Because both inference and training in LSL have explicit budget-control dimensions, it is also necessary to examine the roles of $\text{aug} \times n$, the cycle count C , and the search budget. In general, increasing augmentation or the number of cycles should improve solution quality, but the gains are expected to diminish gradually. Related results can be summarized by a figure showing the effects of augmentation, cycle count, and search budget.

4 Conclusion, limitation, and future work

Conclusion In this work, we have proposed LSL, a closed-loop learning–search framework for large-scale VRP. By combining a globally aware sparse generative backbone with a heuristic refinement backend and multi-solution search feedback, LSL can produce strong search priors, refine them efficiently, and continuously improve its own supervision signal. Extensive comparisons with advanced neural solvers demonstrate LSL’s strong performance on both TSP and CVRP.

Limitation and future work First, although LSL already scales well, the sparse backbone and the search backend could still be co-designed more tightly so that learned priors and downstream local operators interact even more effectively. Second, the current framework uses a fixed family of heuristic refinement operators; learning search control policies or adaptive budget allocation is a promising next step. Third, while the multi-solution soft target already preserves useful uncertainty, richer weighting schemes over search-discovered solutions may further improve the quality of the feedback loop. Finally, extending LSL to a broader range of constrained combinatorial optimization problems remains an important future direction.

References

- [1] Bruce L Golden, S Raghavan, and Edward A Wasil. *The Vehicle Routing Problem: Latest Advances and New Challenges*, volume 43. Springer Science & Business Media, 2008.
- [2] Paolo Toth and Daniele Vigo. *Vehicle routing: problems, methods, and applications*. SIAM, 2014.
- [3] Victor Pillac, Michel Gendreau, Christelle Gu  ret, and Andr  s L Medaglia. A review of dynamic vehicle routing problems. *European Journal of Operational Research*, 225(1):1–11, 2013.
- [4] Grigorios D Konstantakopoulos, Sotiris P Gayialis, and Evripidis P Kechagias. Vehicle routing problem and related algorithms for logistics distribution: A literature review and classification. *Operational research*, 22(3):2033–2062, 2022.
- [5] Giorgio Ausiello, Pierluigi Crescenzi, Giorgio Gambosi, Viggo Kann, Alberto Marchetti-Spaccamela, and Marco Protasi. *Complexity and approximation: Combinatorial optimization problems and their approximability properties*. Springer Science & Business Media, 2012.
- [6] Keld Helsgaun. An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems. *Roskilde: Roskilde University*, 12, 2017.

- [7] Thibaut Vidal. Hybrid genetic search for the cvrp: Open-source implementation and swap* neighborhood. *Computers & Operations Research*, 140:105643, 2022.
- [8] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: a methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.
- [9] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- [10] Irwan Bello, Hieu Pham, Quoc V Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*, 2016.
- [11] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations*, 2019.
- [12] Yeong-Dae Kwon, Jinho Choo, Byoungjip Kim, Iljoo Yoon, Youngjune Gwon, and Seungjai Min. Pomo: Policy optimization with multiple optima for reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 21188–21198, 2020.
- [13] Fu Luo, Xi Lin, Fei Liu, Qingfu Zhang, and Zhenkun Wang. Neural combinatorial optimization with heavy decoder: Toward large scale generalization. In *Advances in Neural Information Processing Systems*, 2023.
- [14] Darko Drakulic, Sofia Michel, Florian Mai, Arnaud Sors, and Jean-Marc Andreoli. BQ-NCO: Bisimulation quotienting for efficient neural combinatorial optimization. In *Advances in Neural Information Processing Systems*, 2023.
- [15] Yan Jin, Yuandong Ding, Xuanhao Pan, Kun He, Li Zhao, Tao Qin, Lei Song, and Jiang Bian. Pointerformer: deep reinforced multi-pointer transformer for the traveling salesman problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
- [16] Jianan Zhou, Yaixin Wu, Wen Song, Zhiguang Cao, and Jie Zhang. Towards omni-generalizable neural methods for vehicle routing problems. In *International Conference on Machine Learning*, pages 42769–42789. PMLR, 2023.
- [17] Yang Wang, Ya-Hui Jia, Wei-Neng Chen, and Yi Mei. Distance-aware attention reshaping: Enhance generalization of neural solver for large-scale vehicle routing problems. *arXiv preprint arXiv:2401.06979*, 2024.
- [18] Changliang Zhou, Xi Lin, Zhenkun Wang, Xialiang Tong, Mingxuan Yuan, and Qingfu Zhang. Instance-conditioned adaptation for large-scale generalization of neural combinatorial optimization. *arXiv preprint arXiv:2405.01906*, 2024.
- [19] Fu Luo, Xi Lin, Yaixin Wu, Zhenkun Wang, Tong Xialiang, Mingxuan Yuan, and Qingfu Zhang. Boosting neural combinatorial optimization for large-scale vehicle routing problems. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=TbTJJNjumY>.
- [20] Yuanyao Chen, Rongsheng Chen, Fu Luo, and Zhenkun Wang. Improving generalization of neural combinatorial optimization for vehicle routing problems via test-time projection learning. In *Advances in Neural Information Processing Systems*, 2025.
- [21] Fu Luo, Xi Lin, Mengyuan Zhong, Fei Liu, Zhenkun Wang, Jianyong Sun, and Qingfu Zhang. Learning to insert for constructive neural vehicle routing solver. In *Advances in Neural Information Processing Systems*, 2025.
- [22] Xuanhao Pan, Yan Jin, Yuandong Ding, Mingxiao Feng, Li Zhao, Lei Song, and Jiang Bian. H-tsp: Hierarchically solving the large-scale traveling salesman problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, February 2023.
- [23] Hanni Cheng, Haosi Zheng, Ya Cong, Weihao Jiang, and Shiliang Pu. Select and optimize: Learning to solve large-scale tsp instances. In *International Conference on Artificial Intelligence and Statistics*, pages 1219–1231. PMLR, 2023.
- [24] Qingchun Hou, Jingwei Yang, Yiqiang Su, Xiaoqing Wang, and Yuming Deng. Generalize learned heuristics to solve large-scale vehicle routing problems in real-time. In *International Conference on Learning Representations*, 2023.
- [25] Haoran Ye, Jiarui Wang, Helan Liang, Zhiguang Cao, Yong Li, and Fanzhang Li. Glop: Learning global partition and local construction for solving large-scale routing problems in real-time. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

- [26] Zhi Zheng, Changliang Zhou, Tong Xialiang, Mingxuan Yuan, and Zhenkun Wang. UDC: A unified neural divide-and-conquer framework for large-scale combinatorial optimization problems. In *Advances in Neural Information Processing Systems*, 2024.
- [27] Chengrui Gao, Haopu Shang, Ke Xue, Dong Li, and Chao Qian. Towards generalizable neural solvers for vehicle routing problems via ensemble with transferrable local policy. In *International Joint Conference on Artificial Intelligence*, 2024.
- [28] Han Fang, Zhihao Song, Paul Weng, and Yutong Ban. INVit: A generalizable routing problem solver with invariant nested view transformer. In *International Conference on Machine Learning*, 2024.
- [29] Chaitanya K Joshi, Thomas Laurent, and Xavier Bresson. An efficient graph convolutional network technique for the travelling salesman problem. *arXiv preprint arXiv:1906.01227*, 2019.
- [30] Zhang-Hua Fu, Kai-Bin Qiu, and Hongyuan Zha. Generalize a small pre-trained model to arbitrarily large tsp instances. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 7474–7482, 2021.
- [31] Ruizhong Qiu, Zhiqing Sun, and Yiming Yang. DIMES: A differentiable meta solver for combinatorial optimization problems. In *Advances in Neural Information Processing Systems*, 2022.
- [32] Zhiqing Sun and Yiming Yang. DIFUSCO: Graph-based diffusion solvers for combinatorial optimization. In *Advances in Neural Information Processing Systems*, 2023.
- [33] Yang Li, Jinpei Guo, Runzhong Wang, and Junchi Yan. From distribution learning in training to gradient search in testing for combinatorial optimization. In *Advances in Neural Information Processing Systems*, 2023.
- [34] Yang Li, Jinpei Guo, Runzhong Wang, Hongyuan Zha, and Junchi Yan. Fast t2t: Optimization consistency speeds up diffusion-based training-to-testing solving for combinatorial optimization. In *Advances in Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=xDrKZ0ZE0c>.
- [35] Yang Li, Lvda Chen, Haonan Wang, Runzhong Wang, and Junchi Yan. Generation as search operator for test-time scaling of diffusion-based combinatorial optimization. In *Advances in Neural Information Processing Systems*, 2025. URL <https://github.com/Thinklab-SJTU/GenSCO>.
- [36] Yining Ma, Zhiguang Cao, and Yeow Meng Chee. Learning to search feasible and infeasible regions of routing problems with flexible neural k-opt. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [37] André Hottung, Paula Wong-Chung, and Kevin Tierney. Neural deconstruction search for vehicle routing problems. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=bCmEP1Ltwq>.
- [38] Liang Xin, Wen Song, Zhiguang Cao, and Jie Zhang. Neurolkh: Combining deep learning model with lin-kernighan-helsgaun heuristic for solving the traveling salesman problem. In *Advances in Neural Information Processing Systems*, volume 34, pages 7472–7483, 2021.
- [39] Yifan Xia, Xianliang Yang, Zichuan Liu, Zhihao Liu, Lei Song, and Jiang Bian. Position: Rethinking post-hoc search-based neural approaches for solving large-scale traveling salesman problems. In *International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=cEJ9jNJJuJP>.
- [40] André Hottung, Yeong-Dae Kwon, and Kevin Tierney. Efficient active search for combinatorial optimization problems. In *International Conference on Learning Representations*, 2022.
- [41] Fu Luo, Xi Lin, Zhenkun Wang, Tong Xialiang, Mingxuan Yuan, and Qingfu Zhang. Self-improved learning for scalable neural combinatorial optimization. *arXiv preprint arXiv:2403.19561*, 2024.
- [42] Jonathan Pirnay and Dominik G. Grimm. Self-improvement for neural combinatorial optimization: Sample without replacement, but improvement. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. Featured Certification.
- [43] Gerhard Reinelt. TSPLIB—a traveling salesman problem library. *ORSA Journal on Computing*, 3(4): 376–384, 1991.
- [44] Eduardo Uchoa, Diego Pecin, Artur Pessoa, Marcus Poggi, Thibaut Vidal, and Anand Subramanian. New benchmark instances for the capacitated vehicle routing problem. *European Journal of Operational Research*, 257(3):845–858, 2017.

- 487 [45] David Applegate, Ribert Bixby, Vasek Chvatal, and William Cook. Concorde tsp solver, 2006.
- 488 [46] Thibaut Vidal. Hybrid genetic search for the cvrp: Open-source implementation and swap* neighborhood.
489 *Computers & Operations Research*, 140:105643, 2022.
- 490 [47] André Hottung, Mridul Mahajan, and Kevin Tierney. Polynet: Learning diverse solution strategies for
491 neural combinatorial optimization. In *International Conference on Learning Representations*, 2025. URL
492 <https://openreview.net/forum?id=TKuYWeFE6S>.
- 493 [48] Yong Liang Goh, Zhiguang Cao, Yining Ma, Jianan Zhou, Mohammed Haroon Dupty, and Wee Sun Lee.
494 Shield: Multi-task multi-distribution vehicle routing solver with sparsity and hierarchy. In *International
495 Conference on Machine Learning*, 2025.
- 496 [49] Dian Meng, Zhiguang Cao, Jie Gao, Yaoxin Wu, and Yaqing Hou. Uniteformer: Unifying node and edge
497 modalities in transformers for vehicle routing problems. In *Advances in Neural Information Processing
498 Systems*, 2025. URL <https://openreview.net/forum?id=BRklmF1CsD>.
- 499 [50] Changliang Zhou, Xi Lin, Zhenkun Wang, and Qingfu Zhang. L2r: Learning to reduce search space for
500 generalizable neural routing solver, 2025. URL <https://arxiv.org/abs/2503.03137>.
- 501 [51] Éric D Taillard and Keld Helsgaun. Popmusic for the travelling salesman problem. *European Journal of
502 Operational Research*, 272(2):420–429, 2019.
- 503 [52] Zhang-Hua Fu, Sipeng Sun, Jintong Ren, Tianshu Yu, Haoyu Zhang, Yuanyuan Liu, Lingxiao Huang,
504 Xiang Yan, and Pinyan Lu. A hierarchical destroy and repair approach for solving very large-scale
505 travelling salesman problem. *arXiv preprint arXiv:2308.04639*, 2023.

A Related work

A.1 Constructive and non-constructive NCO for VRP

Neural combinatorial optimization (NCO) for VRP can be broadly divided into constructive and non-constructive formulations. Constructive methods generate a route or a set of routes sequentially. Pointer Networks [9] establish the classical autoregressive paradigm, and later attention-based solvers substantially improve training stability and solution quality [11, 12]. More recent constructive models strengthen this family through richer decoding, symmetry-aware training, or more flexible action spaces, including BQ-NCO [14], Heavy Decoder / LEHD [13], PolyNet [47], and the recent insertion-based constructive solver L2C-Insert [21]. Their common appeal is that they learn a direct mapping from instances to feasible solutions; however, as the problem scale grows, purely sequential construction still struggles to maintain long-range coordination, correct early mistakes, and preserve global consistency across distant regions or routes.

Non-constructive methods instead predict intermediate structures or iteratively refine candidate solutions. Early heatmap- or score-prediction methods such as Efficient GNN, Att-GCN+MCTS, and DIMES learn edge likelihoods or structural scores and then recover tours with beam search, MCTS, or gradient-based refinement [29–31]. More recent generative methods push this line substantially further. DIFUSCO introduces graph diffusion models for CO and makes diffusion-based edge prediction a competitive neural backbone [32]; T2T then moves from pure distribution learning to objective-guided gradient search at test time over the learned solution distribution [33]; Fast T2T reduces the cost of this paradigm through optimization consistency, enabling higher-quality few-step generation and more efficient test-time refinement [34]; and GenSCO further reframes generation itself as a search operator, alternating solution disruption and generative enhancement to obtain more stable inference-time scaling [35]. This progression is important because non-constructive NCO is no longer limited to one-shot heatmap prediction, but increasingly learns solution distributions that can be actively searched and refined during inference.

A related but distinct branch learns improvement operators or injects neural guidance into strong search heuristics. NeuOpt and Neural Deconstruction Search learn local repair or destroy-and-repair style refinement policies [36, 37], while NeuroLKH augments Lin-Kernighan-Helsgaun with neural candidate guidance [38]. Compared with pure constructive solvers, these methods often offer stronger inference-time correction, but they also depend more heavily on downstream search dynamics. As a result, the central tension on large VRPs remains how to combine scalable computation, global modeling, and effective search feedback without letting any single component dominate the entire solving pipeline.

A.2 Scalable learning methods for large-scale VRP

Scaling learned VRP solvers beyond the small and medium regimes has become a central research topic. Recent work increasingly attacks this challenge through cross-scale generalization and adaptation rather than only through hard decomposition. In particular, Boosting Neural Combinatorial Optimization for Large-Scale Vehicle Routing Problems strengthens constructive large-scale solving with a lightweight linear-complexity backbone and self-improved training on instances up to 100K nodes [19]; Test-Time Projection Learning addresses large-scale distribution shift by learning inference-time projection strategies that align extracted subgraphs with the training distribution without retraining the base solver [20]; and L2C-Insert shows that a stronger constructive action space, namely insertion rather than append-only decoding, can also deliver strong scaling behavior from 100 to 100K nodes [21]. Around these representative 2025 advances, earlier and complementary lines including Pointerformer, omni-generalizable routing solvers, distance-aware attention reshaping, instance-conditioned adaptation, transferable local policies, and INViT progressively improve the inductive bias needed for cross-scale or cross-distribution transfer [15–18, 27, 28].

Another recent route pursues large-scale solving through more efficient generative or search-assisted backbones. DIFUSCO, T2T, Fast T2T, and GenSCO show that diffusion-style solvers need not remain fixed denoising pipelines, but can instead be redesigned for fewer generation steps, more effective test-time search, and stronger inference-time scaling efficiency [32–35]. Although most of these works are developed primarily on TSP or broader CO benchmarks rather than the full spectrum of VRP variants, they are highly relevant to large-scale routing because they shift the focus from maximizing single-pass prediction quality to improving the cost-efficiency of test-time computation.

560 A third line attains scalability through hierarchy, partitioning, or structured locality control. H-TSP,
 561 GLOP, and UDC decompose very large instances into tractable subproblems or reviser stages [22,
 562 25, 26], while recent unified and generalizable solvers such as SHIELD and UniteFormer broaden
 563 the discussion from single-distribution scaling to multi-task, multi-distribution, or multi-modal
 564 generalization [48, 49]. These methods substantially extend the operating range of neural routing, but
 565 many of them obtain scalability or robustness by explicitly constraining locality, partition structure, or
 566 the decision representation. For ultra-large VRP, this trade-off matters because the hardest decisions
 567 often depend on long-range and cross-region interactions rather than only local subproblem quality.

568 **A.3 Hybrid learning + search methods**

569 Hybrid methods combine learned models with explicit search, heuristics, or classical solvers. A
 570 common pattern is to use a neural predictor as a score model or proposal mechanism and then rely
 571 on search to produce the final solution, as in beam-search-based decoding [29], MCTS-style infer-
 572 ence [30, 31], diffusion plus local search [32], training-to-testing gradient search and its accelerated
 573 variants [33, 34], GenSCO-style search-driven generation [35], and neural guidance for LKH [38].
 574 Improvement-oriented hybrids such as NeuOpt and Neural Deconstruction Search further demonstrate
 575 that learned policies can effectively steer local repair or destroy-and-repair processes [36, 37].

576 These hybrids often deliver stronger final performance than pure end-to-end prediction, but in
 577 most cases search still mainly appears as post-processing, decoding, or a one-shot improvement
 578 module. Recent analyses also question whether post-hoc search alone is the right path to large-
 579 scale generalization [39]. In other words, many learning-plus-search methods remain open-loop:
 580 search helps solve the current instance, yet the structural information uncovered during search is not
 581 systematically recycled into future learning. This leaves open the question of how to turn search from
 582 an external solver into a persistent source of learning signal.

583 **A.4 Self-improvement, adaptation, and closed-loop-adjacent methods**

584 Recent work has begun to move from static neural solvers toward adaptive or self-improving pipelines.
 585 In this direction, Luo et al. [19] show that stronger solutions or relabeled data can be fed back into
 586 training to improve scalable routing solvers over time. Instance-conditioned adaptation explores how
 587 a pretrained model can adjust its behavior to each input distribution [18], and L2R uses learning to
 588 reduce the search space by distilling structural guidance that benefits downstream solving [50]. These
 589 directions are important because they move beyond a fixed predictor and recognize that good routing
 590 policies should evolve with the solutions they discover.

591 However, prior self-improvement or adaptation methods typically emphasize only part of the loop.
 592 One layer is that search or adaptation provides structural feedback, better priors, or a more focused
 593 decision space for subsequent inference. A second layer is that search can uncover better-than-original
 594 supervision, enabling iterative self-refinement rather than learning only from the initial dataset. Most
 595 existing works touch one of these layers, but do not fully combine them into a unified “Searching to
 596 Learn” process in which search both reshapes the model’s structural understanding and continuously
 597 produces improved supervision for later rounds.

598 Overall, prior work has laid strong foundations for neural VRP solving from the perspectives of
 599 constructive/non-constructive generation, scalable learning, hybrid search, and self-improvement or
 600 adaptation. LSL is not opposed to these lines, but moves further at their intersection: compared with
 601 methods that obtain scalability mainly through hard decomposition, locality restrictions, or staged
 602 solving, LSL emphasizes preserving global awareness in ultra-large VRP; compared with methods
 603 that treat search as a post-hoc decoder or a one-round refinement module, LSL emphasizes turning
 604 search into a sustained source of structural feedback and stronger supervision; and compared with
 605 methods that cover only one part of the adaptation, relabeling, or self-improvement loop, LSL aims
 606 to unify “Learning to Search” and “Searching to Learn” within the same iterative system framework.

607 B Network architecture details

608 B.1 Unified problem formulation

609 We describe both TSP and CVRP with the same $n \times K$ row-slot sparse graph rather than an explicit
610 $n \times n$ dense graph on the critical path. For each node i , we keep only K candidate slots

$$\mathcal{I}_i = (j_{i,0}, j_{i,1}, \dots, j_{i,K-1}), \quad (16)$$

611 and restrict encoding, decoding, logit prediction, supervision, and downstream refinement to these
612 explicit slots. The candidate-edge set is therefore

$$\mathcal{E}_{\text{cand}} = \{(i, j_{i,r}) \mid i \in \mathcal{V}, r \in \{0, \dots, K-1\}\}. \quad (17)$$

613 The two tasks share the same node-level input abstraction:

$$\mathbf{s}_i = \begin{cases} [\hat{x}_i, \hat{y}_i], & \text{TSP,} \\ [\hat{x}_i, \hat{y}_i, \rho_i], & \text{CVRP,} \end{cases} \quad \rho_i = \begin{cases} d_i/C, & i \neq 0, \\ -1, & i = 0. \end{cases} \quad (18)$$

614 Here $(\hat{x}_i, \hat{y}_i)$ are normalized coordinates. In implementation, we first center the coordinates and then
615 rescale them by the mean node-wise ℓ_2 norm. A TSP solution is a single Hamiltonian tour, whereas a
616 CVRP solution is a set of depot-delimited routes. In both cases, however, the model does not decode
617 a complete route directly; it predicts a sparse structural prior on the same row-slot domain.

618 B.2 Model inputs, noise construction, and shuffle

619 During training, the model receives node features $\{\mathbf{s}_i\}$, the candidate-slot table $\{\mathcal{I}_i\}$, a timestep t ,
620 and a current structural heatmap $\mathcal{H}_t$ defined on the same row-slot domain. The main task-specific
621 difference lies in how $\mathcal{H}_t$ is constructed.

622 **TSP** In TSP, all nodes are ordinary cities, so no depot slot is required. We project the target tour
623 into a row-wise sparse adjacency $Y(\cdot)$ and construct a noisy order σ^{noise} from the target visit order
624 using either a random permutation or several random 2-opt shuffle steps, followed optionally by a
625 global reversal to reduce orientation bias. If target disruption is enabled, we additionally shuffle the
626 teacher order to obtain $\tilde{\sigma}^*$. The TSP decoder state is therefore

$$\mathcal{H}_t^{\text{TSP}} = (1-t)Y(\sigma^{\text{noise}}) + tY(\tilde{\sigma}^*). \quad (19)$$

627 All of these objects are represented directly in the $n \times K$ slot tensor, without introducing any auxiliary
628 $n \times n$ dense adjacency for intermediate structure.

629 **CVRP** For the native CVRP graph, let node 0 be the depot and let n be the number of customers,
630 so the full instance contains $n+1$ nodes. Unlike TSP, we reserve an explicit depot slot for every
631 customer row:

$$\mathcal{I}_i^{\text{CVRP}} = (0, j_{i,1}, j_{i,2}, \dots, j_{i,K-1}), \quad i \in \{1, \dots, n\}. \quad (20)$$

632 That is, slot 0 always corresponds to the depot, while the remaining slots store local customer
633 candidates. The CVRP state contains two components. The first is customer-subgraph noise, built in
634 the same way as in TSP through random permutations, random 2-opt shuffle steps, optional reversal,
635 and optional target disruption. The second is a depot-bias term: from a small set of customers nearest
636 to the depot, we sample a few nodes and create a sparse depot-connection pattern D^{bias} . Thus,

$$\mathcal{H}_{\text{cust}}^{\text{CVRP}} = (1-t)Y(\sigma^{\text{noise}}) + tY(\tilde{\sigma}^*), \quad (21)$$

637

$$\mathcal{H}_{\text{dep}}^{\text{CVRP}} = (1-t)D^{\text{bias}} + tD^*, \quad (22)$$

638

$$\mathcal{H}_t^{\text{CVRP}} = \mathcal{H}_{\text{cust}}^{\text{CVRP}} + \mathcal{H}_{\text{dep}}^{\text{CVRP}}, \quad (23)$$

639 where D^* denotes the target customer-to-depot edge pattern. In this way, route-boundary information
640 is injected into the same sparse slot representation already at the input stage.

641 B.3 Overall architecture: encoder, decoder, and shared backbone

642 The model is best viewed as a coordinate/demand encoder, a timestep-conditioned sparse decoder,
643 and a row-wise sparse-logit head. Given the node input, we first project the raw feature into the
644 hidden space:

$$\mathbf{h}_i^{(0)} = W_{\text{in}} \mathbf{s}_i. \quad (24)$$

645 The encoder then applies multiple sparse multi-head attention layers on the candidate slots $\mathcal{I}_i$,
646 propagating global context with $O(nK)$ rather than $O(n^2)$ complexity, where n denotes the row
647 count of the current row-slot representation:

$$\mathbf{h}_i^{(\ell+1)} = \text{EncLayer}^{(\ell)}\left(\mathbf{h}_i^{(\ell)}, \{\mathbf{h}_{j_{i,r}}^{(\ell)}\}_{r=0}^{K-1}\right). \quad (25)$$

648 The encoded feature is mapped into the decoder space and added with a timestep embedding,

$$\mathbf{z}_i^{(0)} = W_m \mathbf{h}_i + \mathbf{e}(t), \quad (26)$$

649 after which the decoder processes the same row-slot graph while also receiving the current structural
650 heatmap $\mathcal{H}_t$ as an attention bias:

$$\mathbf{z}_i^{(\ell+1)} = \text{DecLayer}^{(\ell)}\left(\mathbf{z}_i^{(\ell)}, \{\mathbf{z}_{j_{i,r}}^{(\ell)}\}_{r=0}^{K-1}; \mathcal{H}_t\right). \quad (27)$$

651 Finally, a lightweight projection head produces the feature used for sparse logit construction,

$$\mathbf{g}_i = \psi(\mathbf{z}_i). \quad (28)$$

652 The encoder therefore aggregates global context over the sparse candidate graph, the decoder injects
653 the timestep and current structural state into the same row-slot domain, and the whole backbone
654 remains sparse-only throughout.

655 B.4 Model outputs and loss functions

656 The model outputs sparse logits on candidate slots rather than complete routes. For node i and slot r ,
657 we compute the score directly as a dot product between the final feature of the source node and that
658 of its candidate node:

$$\ell_{i,r} = \langle \mathbf{g}_i, \mathbf{g}_{j_{i,r}} \rangle, \quad p_{i,r}^{(\theta)} = \frac{\exp(\ell_{i,r})}{\sum_{u=0}^{K-1} \exp(\ell_{i,u})}. \quad (29)$$

659 In implementation, self-loop slots are explicitly masked, and the resulting row-wise sparse probabili-
660 ties are passed directly to the downstream candidate-refinement backend.

661 **TSP** For TSP, we project multiple high-quality tours $\{\tau^{(m)}\}_{m=1}^M$ returned by parallel search onto
662 the same candidate table. For node i , a slot belongs to the support set if it appears as the predecessor
663 or successor of i in at least one retained tour:

$$\mathcal{U}_i^{\text{TSP}} = \{r \mid \exists m, j_{i,r} \in \{\text{prev}_{\tau^{(m)}}(i), \text{next}_{\tau^{(m)}}(i)\}\}. \quad (30)$$

664 The TSP target is therefore the row-wise soft label

$$y_{i,r}^{\text{TSP}} = \begin{cases} \frac{1}{|\mathcal{U}_i^{\text{TSP}}|}, & r \in \mathcal{U}_i^{\text{TSP}}, \\ 0, & \text{otherwise,} \end{cases} \quad \sum_{r=0}^{K-1} y_{i,r}^{\text{TSP}} = 1. \quad (31)$$

665 The corresponding loss is the row-wise sparse cross-entropy,

$$\mathcal{L}_{\text{TSP}} = -\frac{1}{n} \sum_{i=1}^n \sum_{r=0}^{K-1} y_{i,r}^{\text{TSP}} \log p_{i,r}^{(\theta)}. \quad (32)$$

666 Hence, TSP is trained with a multi-solution sparse soft target rather than a single-tour binary label.

667 **CVRP** For CVRP, supervision is again induced by multiple high-quality route sets $\{\mathcal{R}^{(m)}\}_{m=1}^M$,
 668 but the support must explicitly include depot-boundary information:

$$\mathcal{U}_i^{\text{CVRP}} = \left\{ r \mid \begin{array}{l} \exists m, j_{i,r} = \text{prev}_{\mathcal{R}^{(m)}}(i) \\ \text{or } j_{i,r} = \text{next}_{\mathcal{R}^{(m)}}(i) \\ \text{or } (j_{i,r} = 0 \wedge i \text{ is a route-boundary customer}) \end{array} \right\}. \quad (33)$$

669 The CVRP soft target is thus

$$y_{i,r}^{\text{CVRP}} = \begin{cases} \frac{1}{|\mathcal{U}_i^{\text{CVRP}}|}, & r \in \mathcal{U}_i^{\text{CVRP}}, \\ 0, & \text{otherwise,} \end{cases} \quad \sum_{r=0}^{K-1} y_{i,r}^{\text{CVRP}} = 1. \quad (34)$$

670 To emphasize positive customer-to-depot edges, we use a weighted sparse loss:

$$\omega_{i,r}^{\text{CVRP}} = \begin{cases} \lambda_{\text{dep}}, & y_{i,r}^{\text{CVRP}} > 0 \text{ and } j_{i,r} = 0, \\ 1, & \text{otherwise,} \end{cases} \quad (35)$$

671

$$\mathcal{L}_{\text{CVRP}} = -\frac{1}{n} \sum_{i=1}^n \sum_{r=0}^{K-1} \omega_{i,r}^{\text{CVRP}} y_{i,r}^{\text{CVRP}} \log p_{i,r}^{(\theta)}. \quad (36)$$

672 In addition, the depot-slot probability is reused at inference time to estimate boundary customers and
 673 decide how many depot copies should be instantiated inside the sparse candidate graph. At no point
 674 do we fall back to a dense $(N + E) \times (N + E)$ score matrix.

675 B.5 Sparse multi-head attention Triton kernel and online softmax

676 In implementation, both the sparse encoder and the sparse decoder call the same sparse multi-head
 677 attention interface. For head h , row i , and slot r , we define

$$\mathbf{q}_i^{(h)} = W_Q^{(h)} \mathbf{z}_i, \quad \mathbf{k}_{i,r}^{(h)} = W_K^{(h)} \mathbf{z}_{j_{i,r}}, \quad \mathbf{v}_{i,r}^{(h)} = W_V^{(h)} \mathbf{z}_{j_{i,r}}, \quad (37)$$

678

$$e_{i,r}^{(h)} = \frac{\langle \mathbf{q}_i^{(h)}, \mathbf{k}_{i,r}^{(h)} \rangle}{\sqrt{d_h}} + b_{i,r}^{(h)}. \quad (38)$$

679 The bias term $b_{i,r}^{(h)}$ absorbs invalid-slot masks, current-state biases, and task-specific structural biases.
 680 Instead of materializing a full row softmax, the Triton kernel maintains a running maximum and a
 681 running partition sum over slot blocks:

$$m_i^{(h,t)} = \max \left(m_i^{(h,t-1)}, \max_{r \in \mathcal{B}_t} \hat{e}_{i,r}^{(h,t)} \right), \quad m_i^{(h,0)} = -\infty, \quad (39)$$

682

$$\lambda_i^{(h,t)} = \lambda_i^{(h,t-1)} \exp(m_i^{(h,t-1)} - m_i^{(h,t)}) + \sum_{r \in \mathcal{B}_t} \exp(\hat{e}_{i,r}^{(h,t)} - m_i^{(h,t)}), \quad \lambda_i^{(h,0)} = 0. \quad (40)$$

683 Here $\hat{e}_{i,r}^{(h,t)}$ denotes the masked score inside block $\mathcal{B}_t$. Using the same online-softmax statistics, the
 684 kernel updates the value aggregation in one pass:

$$\mathbf{o}_i^{(h,t)} = \frac{\lambda_i^{(h,t-1)} \exp(m_i^{(h,t-1)} - m_i^{(h,t)})}{\lambda_i^{(h,t)}} \mathbf{o}_i^{(h,t-1)} + \frac{1}{\lambda_i^{(h,t)}} \sum_{r \in \mathcal{B}_t} \exp(\hat{e}_{i,r}^{(h,t)} - m_i^{(h,t)}) \mathbf{v}_{i,r}^{(h)}. \quad (41)$$

685 The final normalized attention weight is

$$\alpha_{i,r}^{(h)} = \frac{\exp(e_{i,r}^{(h)} - m_i^{(h,T)})}{\lambda_i^{(h,T)}}. \quad (42)$$

686 Compared with the JAX reference implementation, the Triton version further fuses several adjacent
 687 query rows and multiple attention heads inside one kernel, sharing the reads of ‘neighbor_idx’ and
 688 bias. We also implement dedicated Triton kernels for sparse logits and sparse cross-entropy, so
 689 gather-dot, row-wise normalization, and loss computation all stay inside the same $n \times K$ slot domain.
 690 This is the key systems ingredient that makes the sparse-only backbone efficient in practice.

C Implementation details of baselines

This section only documents the baselines that actually appear in Tables 1, 2, and 3. They include **(1) Classical Solvers:** Concorde [45], LKH3 [6], and HGS [46]; **(2) Constructive NCO Methods:** POMO [12], BQ [14], LEHD [13], INVIT [28], ELG [27], and the two L2C-Insert variants (Greedy and $I=1000$); **(3) Heatmap-based NCO Methods:** Att-GCN+MCTS [30], DIFUSCO [32], T2T [33], Fast-T2T, and GenSCO; and **(4) Two-stage Methods:** H-TSP [22], GLOP [25], GLOP-G (LKH3), and UDC, where Tables 1 and 2 use UDC- $\mathbf{x}_{50}$ while Table 3 uses UDC- $\mathbf{x}_{250}$. Methods that do not appear in these three main tables are omitted here.

For the starred entries in Tables 1 and 2, we directly report the published numbers from the corresponding papers. Unless otherwise stated, the remaining baselines are run with public implementations under the default evaluation settings from their papers or repositories; the method names shown in the tables (e.g., “POMO aug $\times 8$ ”, “BQ bs16”, “LEHD RRC1000”, “INVIT greedy”, “L2C-Insert Greedy”, and “L2C-Insert ($I=1000$)”) directly indicate the inference configuration used.

- **Concorde** [45] We run the Python wrapper PyConcorde with default settings across TSP instances of different sizes.
- **LKH3** [6] We run the code provided by AM [11]. For TSP100K, we use POPMUSIC [51] to reduce the complexity for pre-processing (by setting CANDIDATE SET TYPE = POPMUSIC and INITIAL PERIOD=1000), following the suggestion from Fu et al. [52]. Other settings are default.
- **HGS** [46] We run the Python wrapper for the HGS-CVRP solver, namely PyHygese.
- **LEHD** [13] We limited the maximum destruction size of RRC to 1000 after observing that performance remained nearly unchanged when compared to cases without this restriction, while inference time increased significantly.
- **L2C-Insert** We report both the greedy decoding variant and the version with local reconstruction using $I=1000$, denoted in the tables as “L2C-Insert Greedy” and “L2C-Insert ($I=1000$)”.
- **UDC** We keep the variant naming from the source paper: UDC- $\mathbf{x}_{50}$ ($\alpha=50$) in the synthetic-data main tables and UDC- $\mathbf{x}_{250}$ ($\alpha=50$) in the real-world main table.

D Supplementary experiments

This section reports supplementary materials. We first provide supplementary real-world TSP results on TSPLIB, VLSI, and Art-TSP, together with real-world CVRP results on CVRPLIB. For TSPLIB, we transcribe the baseline columns from GenSCO [35] and further add LSL gap and Time(s) columns under the Guided-LKH t512 setting; for VLSI, we organize two additional tables for the < 2000 and $2000\text{--}200\text{K}$ size ranges; for Art-TSP, we additionally report Guided-LKH results on ultra-large real-world instances together with local L2C-Insert results; for CVRPLIB, we list the remaining instance names in the local instance directory after removing the ORTEC, Loggi, and Belgium extension sets and split them into the < 2000 and $2000\text{--}20\text{K}$ ranges. The later appendix sections then provide further methodological discussion, complexity analysis, and representative visualizations.

D.1 Real-world TSP results on TSPLIB

Tables 5 and 6 report supplementary real-world TSPLIB TSP results. The AM*, GCN*, DIFUSCO, Fast T2T, and GenSCO baseline columns are transcribed from Tables 7 and 8 of GenSCO [35]; the added L2C-Insert column comes from our local evaluation results, while the NeuroLKH column is transcribed from the supplementary TSPLIB tables of NeuroLKH [38]; the last two columns report our LSL gap and Time(s) under the Guided-LKH t512 setting. On the 29 instances with $50\text{--}200$ nodes, LSL achieves an average gap of 0.000% with an average Time(s) of 11.77 . For the larger $200\text{--}1000$ -node group, the table contains 19 instances; LSL then achieves an average gap of 0.008% with an average Time(s) of 23.99 , with only small nonzero gaps on f1417, u574, and p654 (0.035% , 0.093% , and 0.025% , respectively). Overall, Time(s) increases with instance size as expected, while solution quality remains near-optimal under the t512 setting.

Table 5: Results on real-world TSPLIB TSP instances with 50–200 nodes. The AM*, GCN*, DIFUSCO, Fast T2T, and GenSCO columns are transcribed from Table 7 of GenSCO [35]. “*” denotes results quoted from previous works in the source paper. The last two columns report our LSL results under the Guided-LKH t512 setting.

Instances	AM*	GCN*	DIFUSCO	Fast T2T	GenSCO	L2C-Insert	NeuroLKH	LSL Gap	LSL Time(s)
eil51	16.767%	40.025%	2.82%	0.00%	0.000%	0.000%	0.000%	0.000%	7.44
berlin52	4.169%	33.225%	0.00%	0.00%	0.000%	1.584%	0.000%	0.000%	7.82
st70	1.737%	24.785%	0.00%	0.00%	0.000%	0.000%	0.000%	0.000%	8.73
eil76	1.992%	27.411%	0.34%	0.00%	0.000%	0.004%	0.000%	0.000%	8.69
pr76	0.816%	27.793%	1.12%	0.00%	0.000%	0.000%	0.000%	0.000%	8.25
rat99	2.645%	17.633%	0.09%	0.00%	0.000%	12.416%	0.000%	0.000%	8.63
kroA100	4.017%	28.828%	0.10%	0.00%	0.000%	5.013%	0.000%	0.000%	8.60
kroB100	5.142%	34.686%	2.29%	0.65%	0.000%	5.300%	0.000%	0.000%	8.62
kroC100	0.972%	35.506%	0.00%	0.00%	0.000%	6.076%	0.000%	0.000%	8.59
kroD100	2.717%	38.018%	0.07%	0.00%	0.000%	4.815%	0.000%	0.000%	8.79
kroE100	1.470%	26.589%	3.83%	0.00%	0.000%	4.050%	0.000%	0.000%	8.75
rd100	3.407%	50.432%	0.08%	0.00%	0.000%	0.000%	0.000%	0.000%	8.53
eil101	2.994%	26.701%	0.03%	0.00%	0.000%	0.000%	0.000%	0.000%	8.30
lin105	1.739%	34.902%	0.00%	0.00%	0.000%	4.337%	0.000%	0.000%	7.88
pr107	3.933%	80.564%	0.91%	0.62%	0.000%	0.000%	0.000%	0.000%	9.31
pr124	3.677%	70.146%	1.02%	0.08%	0.000%	0.000%	0.000%	0.000%	10.06
bier127	5.908%	45.561%	0.94%	1.50%	0.000%	0.000%	0.016%	0.000%	10.17
ch130	3.182%	39.090%	0.29%	0.00%	0.000%	0.000%	0.000%	0.000%	11.64
pr136	5.064%	58.673%	0.19%	0.01%	0.000%	0.000%	0.000%	0.000%	13.86
pr144	7.641%	55.837%	0.80%	0.39%	0.000%	0.000%	0.081%	0.000%	15.66
ch150	4.584%	49.743%	0.57%	0.00%	0.000%	0.000%	0.000%	0.000%	11.69
kroA150	3.784%	45.411%	0.34%	0.00%	0.000%	3.220%	0.000%	0.000%	11.86
kroB150	2.437%	56.745%	0.30%	0.07%	0.340%	3.846%	0.000%	0.000%	12.75
pr152	7.494%	33.925%	1.69%	0.19%	0.187%	0.005%	0.037%	0.000%	30.35
u159	7.551%	38.338%	0.82%	0.00%	0.000%	0.025%	0.000%	0.000%	11.40
rat195	6.893%	24.968%	1.48%	0.79%	0.194%	4.828%	0.000%	0.000%	14.72
d198	373.020%	62.351%	3.32%	0.86%	0.751%	1.835%	0.285%	0.000%	35.11
kroA200	7.106%	40.885%	2.28%	0.49%	0.160%	1.025%	0.000%	0.000%	12.67
kroB200	8.541%	43.643%	2.35%	2.50%	0.098%	0.219%	0.000%	0.000%	12.39
Mean	16.767%	40.025%	0.97%	0.28%	0.061%	2.021%	0.014%	0.000%	11.77

D.2 Real-world TSP results on VLSI

Tables 7 and 8 report supplementary real-world VLSI TSP results, both transcribed from the corresponding real-world result summary file. The former covers instances with fewer than 2000 nodes and reports GenSCO, L2C-Insert, NeuroLKH, LSL Gap, and LSL Time(s). The latter covers instances with 2000–200K nodes and reports L2C-Insert, NeuroLKH, LSL Gap, and LSL Time(s).

D.3 Real-world TSP results on Art-TSP

Table 9 reports results on the 6 ultra-large real-world TSP instances from Art-TSP. Across these 6 instances, LSL keeps the gap within 0.025%–0.032%, and the wall-clock Time(s) increases from 12105.279 at 100K nodes to 24469.262 at 200K nodes.

Table 6: Results on real-world TSPLIB TSP instances with 200–1000 nodes. The DIFUSCO, Fast T2T, and GenSCO columns are transcribed from Table 8 of GenSCO [35]. The last two columns report our LSL results under the Guided-LKH t512 setting.

Instances	DIFUSCO	Fast T2T	GenSCO	L2C-Insert	NeuroLKH	LSL Gap	LSL Time(s)
a280	1.39%	0.10%	0.000%	0.109%	0.000%	0.000%	18.76
d493	1.81%	1.43%	3.872%	0.469%	0.086%	0.000%	21.35
d657	4.86%	0.64%	0.457%	0.751%	0.001%	0.000%	20.16
fl417	3.30%	2.01%	0.596%	0.408%	0.056%	0.035%	46.46
gil262	2.18%	0.18%	0.000%	0.089%	0.000%	0.000%	13.53
lin318	2.95%	1.21%	0.025%	0.038%	0.000%	0.000%	13.96
p654	7.49%	1.67%	1.023%	1.815%	0.354%	0.025%	96.24
pcb442	2.59%	0.61%	0.015%	0.598%	0.000%	0.000%	15.95
pr226	4.22%	0.34%	0.211%	0.290%	0.016%	0.000%	14.11
pr264	0.92%	0.73%	0.000%	0.000%	0.000%	0.000%	35.31
pr299	1.46%	1.40%	0.011%	2.390%	0.000%	0.000%	13.90
pr439	2.73%	0.50%	0.106%	0.387%	0.047%	0.000%	22.11
rat575	2.32%	1.43%	0.000%	0.825%	0.001%	0.000%	18.61
rat783	3.04%	1.03%	0.068%	0.995%	0.000%	0.000%	20.82
rd400	1.18%	0.08%	0.030%	0.162%	0.000%	0.000%	14.52
ts225	4.95%	1.37%	0.517%	0.000%	0.000%	0.000%	12.48
tsp225	3.25%	0.81%	0.000%	2.860%	0.000%	0.000%	12.10
u574	2.50%	0.94%	0.109%	0.630%	0.000%	0.093%	17.05
u724	2.05%	1.41%	0.040%	0.555%	0.000%	0.000%	28.36
Mean	2.905%	0.942%	0.373%	0.704%	0.030%	0.008%	23.99

Table 7: Results on real-world VLSI TSP instances with fewer than 2000 nodes.

Instance	GenSCO	L2C-Insert	NeuroLKH	LSL Gap	LSL Time(s)	Instance	GenSCO	L2C-Insert	NeuroLKH	LSL Gap	LSL Time(s)
xqf131	-0.1378%	4.7647%	0.00%			pbd984	-0.2248%	0.3784%	0.00%		
xqg237	0.0431%	1.2889%	0.00%			xit1083	-0.2262%	0.6737%	0.00%		
pma343	0.6107%	15.2290%	0.00%			dka1376	-0.1229%	1.6946%	0.00%		
pka379	1.7708%	19.8753%	0.00%			dca1389	0.4114%	0.5395%	0.00%		
bcl380	-0.1926%	-0.1342%	0.00%			dja1436	0.4369%	1.0077%	0.00%		
pbl395	-0.3638%	-0.3565%	0.00%			icw1483	0.2263%	1.2454%	0.00%		
pbk411	-0.4599%	-0.3756%	0.00%			fra1488	3.3967%	0.6374%	0.00%		
pbn423	-0.3832%	-0.3395%	0.00%			rbv1583	0.5977%	1.1684%	0.00%		
pbn436	-0.3636%	0.1982%	0.00%			rby1599	0.6602%	0.9665%	0.00%		
xql662	-0.1125%	0.0279%	0.00%			fnb1615	1031.5743%	1.0392%	0.02%		
rbx711	-0.1289%	0.9480%	0.00%			djc1785	2.2686%	0.9319%	0.00%		
rbu737	0.2230%	0.7630%	0.00%			dcc1911	0.7610%	1.1018%	0.02%		
dkg813	0.8194%	0.6660%	0.00%			dcd1973	23.2987%	0.6533%	0.05%		
lim963	-0.0551%	0.7854%	0.00%								

Table 8: Results on real-world VLSI TSP instances with 2000–200K nodes.

Instance	L2C-Insert	NeuroLKH	LSL Gap	LSL Time(s)	Instance	L2C-Insert	NeuroLKH	LSL Gap	LSL Time(s)
djb2036	0.4914%	0.00%	-0.331%	359.691	ido21215	3.4863%		-0.261%	1359.514
dcb2086	1.0685%	0.00%	-0.192%	338.146	fma21553	3.7856%		-0.237%	1247.462
bva2144	0.8540%	0.00%	-0.221%	433.101	lsb22777	4.2038%		-0.291%	1376.764
xqc2175	1.5891%	0.00%	-0.214%	496.137	xrh24104	5.3299%		-0.242%	2226.190
bck2217	0.7445%	0.01%	-0.330%	1387.759	bbz25234	4.9082%		-0.318%	2102.744
xpr2308	1.2130%	0.00%	-0.239%	1147.541	irx28268	4.2582%		-0.363%	2092.812
ley2323	1.1626%	0.10%	-0.233%	1147.441	fyg28534	7.7193%		-0.282%	1937.243
irw2802	1.5471%	0.00%	-0.253%	1147.305	icx28698	6.6384%		-0.294%	1937.086
dbj2924	1.3585%	0.00%	-0.137%	1147.130	boa28924	6.4090%		-0.295%	1612.595
pia3056	2.2171%	0.00%	-0.339%	942.189	ird29514	5.4607%		-0.225%	1803.827
xqc3891	1.2462%	0.02%	-0.183%	1040.456	pbb30440	4.9287%		-0.186%	1244.193
bgb4355	1.4532%	0.06%	-0.333%	930.654	xib32892	5.4544%		-0.257%	1555.981
xsc6880	2.0405%	0.02%	-0.374%	1246.073	fry33203	7.1948%		-0.187%	4260.619
lap7454	3.0127%	0.05%	-0.328%	1226.154	bby34656	6.3148%		-0.291%	4260.516
ida8197	2.1110%	0.16%	-0.300%	2084.140	pba38478	6.5314%		-0.260%	3958.797
xmc10150	2.4490%	0.10%	-0.295%	2835.700	ics39603	9.4781%		-0.315%	4115.049
xvb13584	2.3437%		-0.276%	2956.059	rbz43748	7.4829%		-0.309%	4114.862
xrb14233	3.2484%		-0.120%	2380.095	tht47608	8.6438%		-0.261%	3958.690
xia16928	3.7517%		0.197%	3310.673	fna52057	7.3344%		-0.276%	4260.355
pjh17845	3.0073%		0.023%	3077.562	bna56769	6.7615%		-0.331%	3958.543
frh19289	3.9645%		-0.102%	2955.976	dan59296	9.1025%		-0.249%	3783.674
fnc19402	3.8160%		0.020%	2276.261	sra104815	6.1138%		-0.701%	9706.833

Table 9: Results on ultra-large real-world TSP instances from Art-TSP.

Instance	L2C-Insert	NeuroLKH	LSL Gap (%)	LSL Time(s)
mona-lisa100K	3.7640%		0.029338	12105.279
vangogh120K	4.0220%		0.031460	14898.313
venus140K	4.1016%		0.025215	17860.376
pareja160K	4.2593%		0.026748	19513.381
courbet180K	4.7775%		0.027445	22618.978
earring200K	4.5136%		0.031780	24469.262

749 **D.4 Real-world CVRP results on CVRPLIB**

750 Tables 10 and 11 report supplementary real-world CVRPLIB CVRP results. The instance names are
751 taken directly from the `.vrp` filenames under the local `test_dataset/cvrp/cvrplib/instances`
752 directory after removing the ORTEC, Loggi, and Belgium extension sets. Table 10 now lists only the
753 100 `set-X` instances (from `X-n101-k25` to `X-n1001-k43`), while Table 11 collects all 100 `set-XL`
754 instances (from `XL-n1048-k237` to `XL-n10001-k1570`), including the sub-2000-node `set-XL` cases
755 previously mixed into the smaller table. Following the current supplementary comparison layout,
756 we keep L2C-Insert, NeuroLKH, LSL Gap, and LSL Time(s) columns for every instance. To reduce
757 vertical length without changing the page orientation, both tables use a three-panel side-by-side layout
758 on standard portrait pages with a font size aligned to the TSPLIB 50–200-node table and tighter
759 column spacing.

Table 10: Results on real-world CVRPLIB set-X CVRP instances.

Instance	L2C-Insert	Neuro LKH	LSL Gap	LSL Time(s)	Instance	L2C-Insert	Neuro LKH	LSL Gap	LSL Time(s)	Instance	L2C-Insert	Neuro LKH	LSL Gap	LSL Time(s)
X-n101-k25		0.722%			X-n261-k13		2.825%			X-n491-k59				
X-n106-k14		0.632%			X-n266-k58		1.464%			X-n502-k39				
X-n110-k13		0.689%			X-n270-k35		1.492%			X-n513-k21				
X-n115-k10		0.289%			X-n275-k28		1.257%			X-n524-k153				
X-n120-k6		0.433%			X-n280-k17		2.016%			X-n536-k96				
X-n125-k30		0.797%			X-n284-k15		3.084%			X-n548-k50				
X-n129-k18		0.982%			X-n289-k60		1.942%			X-n561-k42				
X-n134-k13		1.178%			X-n294-k50		1.598%			X-n573-k30				
X-n139-k10		0.829%			X-n298-k31		2.578%			X-n586-k159				
X-n143-k7		1.184%			X-n303-k21					X-n599-k92				
X-n148-k46		0.852%			X-n308-k13					X-n613-k62				
X-n153-k22		0.429%			X-n313-k71					X-n627-k43				
X-n157-k13		0.171%			X-n317-k53					X-n641-k35				
X-n162-k11		0.759%			X-n322-k28					X-n655-k131				
X-n167-k10		1.463%			X-n327-k20					X-n670-k130				
X-n172-k51		0.850%			X-n331-k15					X-n685-k75				
X-n176-k26		0.978%			X-n336-k84					X-n701-k44				
X-n181-k23		0.457%			X-n344-k43					X-n716-k35				
X-n186-k15		1.566%			X-n351-k40					X-n733-k159				
X-n190-k8		1.648%			X-n359-k29					X-n749-k98				
X-n195-k51		1.047%			X-n367-k17					X-n766-k71				
X-n200-k36		0.899%			X-n376-k94					X-n783-k48				
X-n204-k19		1.496%			X-n384-k52					X-n801-k40				
X-n209-k16		1.579%			X-n393-k38					X-n819-k171				
X-n214-k11		5.916%			X-n401-k29					X-n837-k142				
X-n219-k73		0.118%			X-n411-k19					X-n856-k95				
X-n223-k34		1.449%			X-n420-k130					X-n876-k59				
X-n228-k23		1.121%			X-n429-k61					X-n895-k37				
X-n233-k16		1.911%			X-n439-k37					X-n916-k207				
X-n237-k14		1.334%			X-n449-k29					X-n936-k151				
X-n242-k48		1.200%			X-n459-k26					X-n957-k87				
X-n247-k50		0.979%			X-n469-k138					X-n979-k58				
X-n251-k28		1.488%			X-n480-k70					X-n1001-k43				
X-n256-k16		3.158%												

Table 11: Results on real-world CVRPLIB set-XL CVRP instances.

Instance	L2C-Insert	Neuro LKH	LSL Gap	LSL Time(s)	Instance	L2C-Insert	Neuro LKH	LSL Gap	LSL Time(s)	Instance	L2C-Insert	Neuro LKH	LSL Gap	LSL Time(s)
XL-n1048-k237					XL-n2634-k17					XL-n4951-k203				
XL-n1094-k157					XL-n2681-k540					XL-n5061-k184				
XL-n1141-k112					XL-n2727-k546					XL-n5174-k55				
XL-n1188-k96					XL-n2774-k286					XL-n5288-k1246				
XL-n1234-k55					XL-n2821-k208					XL-n5406-k783				
XL-n1281-k29					XL-n2867-k120					XL-n5526-k553				
XL-n1328-k19					XL-n2914-k95					XL-n5649-k401				
XL-n1374-k278					XL-n2961-k55					XL-n5774-k290				
XL-n1421-k232					XL-n3007-k658					XL-n5902-k122				
XL-n1468-k151					XL-n3054-k461					XL-n6034-k61				
XL-n1514-k106					XL-n3101-k311					XL-n6168-k1922				
XL-n1561-k75					XL-n3147-k232					XL-n6305-k1042				
XL-n1608-k39					XL-n3194-k161					XL-n6445-k628				
XL-n1654-k11					XL-n3241-k115					XL-n6588-k473				
XL-n1701-k562					XL-n3287-k30					XL-n6734-k330				
XL-n1748-k271					XL-n3334-k934					XL-n6884-k148				
XL-n1794-k163					XL-n3408-k524					XL-n7037-k38				
XL-n1841-k126					XL-n3484-k436					XL-n7193-k1683				
XL-n1888-k82					XL-n3561-k229					XL-n7353-k1471				
XL-n1934-k46					XL-n3640-k211					XL-n7516-k859				
XL-n1981-k13					XL-n3721-k77					XL-n7683-k602				
XL-n2028-k617					XL-n3804-k29					XL-n7854-k365				
XL-n2074-k264					XL-n3888-k1010					XL-n8028-k294				
XL-n2121-k186					XL-n3975-k687					XL-n8207-k108				
XL-n2168-k138					XL-n4063-k347					XL-n8389-k2028				
XL-n2214-k131					XL-n4153-k291					XL-n8575-k1297				
XL-n2261-k54					XL-n4245-k203					XL-n8766-k1032				
XL-n2307-k34					XL-n4340-k148					XL-n8960-k634				
XL-n2354-k631					XL-n4436-k48					XL-n9160-k379				
XL-n2401-k408					XL-n4535-k1134					XL-n9363-k209				
XL-n2447-k290					XL-n4635-k790					XL-n9571-k55				
XL-n2494-k194					XL-n4738-k487					XL-n9784-k2774				
XL-n2541-k121					XL-n4844-k321					XL-n10001-k1570				
XL-n2587-k66														

E Why LSL separates global prediction from discrete search

The reported performance of LSL comes from the interaction between two components: a globally aware sparse predictor and a discrete heuristic refinement backend. The key design choice is that these two responsibilities are separated rather than fused into one heavy constructive decoder.

Why not put everything into the decoder Older constructive paradigms often ask the neural model to do three things at once during solution construction: infer global structure, satisfy combinatorial constraints, and make fine-grained discrete placement decisions. At the same time, many existing methods are still trained mainly against a single reference solution or an almost one-hot supervision target. On small instances, such targets can still work reasonably well because high-quality or even optimal solutions are comparatively easier to obtain, so the model can implicitly absorb a preference for lower-cost structures from these sharp labels.

The situation changes on large instances. High-quality supervision becomes much harder to obtain, and the training set is more likely to contain suboptimal or outlier solutions whose quality is difficult to identify explicitly. If training still relies on a single-solution, highly concentrated target, these conflicting signals are pushed directly into the model, which can both weaken training and force the model to commit too early to one particular route pattern. Meanwhile, as the partial solution becomes more complex, the decoder must also reason over unresolved future nodes, long-range geometric interactions, and feasibility constraints, which further amplifies the computational and learning burden of purely neural constructive decoding.

Why LSL is different LSL avoids this bottleneck by making the sparse backbone responsible for globally informed candidate scoring, while the heuristic backend handles discrete feasibility and iterative improvement. The model is no longer required to emit a final feasible solution in one shot, nor does it need to compress both low-cost preference learning and full constraint satisfaction into a single decoder. Instead, the neural module only needs to output a structural prior that is genuinely useful for downstream search, and the discrete backend turns this prior into feasible and progressively improved solutions.

This decomposition also prevents the search space from being locked too early by single-solution supervision. During training, LSL does not force every node to match one unique “correct” edge. Instead, multiple high-quality search solutions are aggregated into a row-wise soft target. As a result, the model can become sharp where local structure is stable, while preserving uncertainty where several local alternatives remain plausible. From the perspective of search, this means the system is not forced to represent every intermediate step as the one final feasible route encoded by the supervision target; it can refine more flexibly before returning to a stronger solution. This is precisely why LSL scales better on large instances: the model does not need to solve the whole combinatorial problem inside an autoregressive constructive process, yet the search backend still receives a strong learned prior that is much more search-friendly.

F Clarification on the row-wise soft target support and the cross-entropy loss

We further clarify how the supervision target used by LSL is defined and why the row-wise cross-entropy in Equation 14 is a principled choice.

The target is not a single edge per node For each training instance, we retain multiple high-quality solutions obtained by parallel heuristic search and project them onto the same sparse candidate graph. For a node i , the target support consists of all candidate edges that appear in at least one retained high-quality solution. Therefore, the supervision for node i is not collapsed to one “correct” candidate edge.

How the row-wise soft target is formed Let $\mathcal{U}_i$ denote the set of candidate edges incident to node i that are observed in at least one retained solution after projection onto the candidate graph. We define the row-wise soft target by assigning uniform mass to every edge in $\mathcal{U}_i$ and zero mass to the remaining candidates, followed by row normalization. As a result, the target distribution becomes

sharp when many high-quality solutions agree on the same local structure, and remains diffuse when multiple local alternatives persist.

Why this support definition matters This construction preserves the structural uncertainty revealed by search. The learned prior therefore tells the downstream solver both where the model should be confident and where alternative local moves should remain available for later refinement.

Why the row-wise cross-entropy is reasonable The loss in Equation 14 is a row-wise cross-entropy between the predicted sparse candidate distribution and the soft target induced by multiple high-quality search solutions. This is a principled choice because the model output for each node is itself a probability distribution over the candidate set.

Unlike a hard one-hot target, the soft target used by LSL may place probability mass on multiple candidate edges. The row-wise cross-entropy therefore encourages two behaviors simultaneously: it sharpens the model when search repeatedly confirms the same local structure, and it preserves ambiguity when search outcomes consistently reveal several plausible alternatives.

This objective is also well suited to gradient-based optimization. Candidate edges that the model assigns too little probability despite repeated support from search incur a larger penalty, whereas edges in genuinely ambiguous regions are not forced into an artificially over-confident one-hot target. In this way, the loss aligns the training objective with the role of the model at inference time: producing a search-friendly sparse prior rather than a prematurely collapsed final route.

G Clarification on fair comparison and calculating costs

A note on comparing computational budgets We would like to clarify that directly comparing iteration counts is difficult, because the definition and computational cost of one iteration differ substantially across methods:

- Improvement-based methods such as NeurOpt perform very fast local-search operations (e.g., k-opt) in each iteration.
- Two-stage methods such as GLOP and UDC call multiple sub-problem solvers per iteration, or perform destroy-and-repair on multiple initial solutions, and may also invoke a strong heuristic solver such as LKH3 internally.
- LSL performs refinement on a single sparse-prior-guided solution trajectory using one neural backbone and one heuristic backend.

Therefore, total inference time may provide a more comprehensive and fair reflection of computational budget. Based on this view, our analysis focuses on the solution quality (optimality gap) achieved within a given runtime. From the perspective of total runtime, our experimental results support this claim. For example, in Table 1 (TSP1K), LSL ($I=1000$) achieves a 0.481% gap in 46.4 minutes. By contrast, a strong baseline such as LEHD+RRC1000 requires 5.2 hours to reach a 0.731% gap. This suggests that our method provides a favorable trade-off between runtime and solution quality.

H Total inference time and average inference time per instance

The apparent decrease in total inference time for larger instances is due to the different number of instances in the test datasets for each problem size.

Following the standard evaluation protocols established in prior NCO research [13, 30], the test sets for larger problems contain significantly fewer instances. For example, our TSP100 test set contains 10,000 instances, while the TSP10K and TSP100K sets contain only 16 instances each. The total inference times reported in Tables 1 and 2 of our main paper reflect the time required to solve the entire dataset.

To provide a clearer picture, the table below breaks down the total inference time into the average time per instance.

As the table clearly shows, the average inference time per instance for LSL scales with the problem size as expected, increasing from 2.9 seconds for TSP100 to 97.8 seconds for TSP100K.

Table 12: Total inference time and average inference time per instance of LSL.

Dataset Size	TSP100 10000 instances	TSP1K 128 instances	TSP10K 16 instances	TSP100K 16 instances
LSL Total Inference Time	8.0h	46.4m	16.64m	26.07m
LSL Inference Time per Instance	2.9s	21.8s	62.4s	97.8s

I LSL method positioning and core contributions

To avoid confusion with prior learning-for-search or post-hoc refinement pipelines, we further clarify the positioning of LSL and the core contributions of this paper. In the most direct terms, our goal is not to build a loose two-stage pipeline in which a neural model produces a heatmap and a search heuristic simply cleans it up afterward. Instead, we aim to establish a closed-loop system for large-scale VRP in which learning provides search-friendly priors and search, in turn, continually improves the learning target itself.

What problem does LSL solve? The main challenge addressed here is how to achieve scalability, global awareness, and strong solution quality *simultaneously* on ultra-large VRP. Many learning-based solvers become too expensive in memory or computation as the problem size grows, while scalable alternatives often rely on partitioning or staged decomposition that break long-range interactions. At the same time, many existing learning-plus-search methods still remain open-loop: search improves the current instance, but its outputs are rarely turned into persistent supervision for future rounds of learning.

What is the core positioning of LSL? LSL is a closed-loop framework in which *Learning to Search* and *Searching to Learn* reinforce each other. Learning to Search means that the model is trained to output structural priors that are useful for downstream search rather than a one-shot final route. Searching to Learn has two meanings. First, search feeds structural feedback back to the model. Second, and more importantly, search does not return only a single improved route. We retain multiple high-quality solutions obtained by parallel heuristic search and project them onto the same sparse candidate graph to build a row-wise soft target distribution. This target is concentrated on structurally stable nodes and remains diffuse on uncertain nodes, so the model learns where to be confident and where to preserve alternatives for later search.

Why was such a closed loop hard to realize before? One obstacle is architectural: if the backbone itself cannot scale, the loop cannot run on truly large instances; if scalability is obtained only through partitioning, the learned prior already loses global information before search begins. This is why we design a lightweight globally-aware sparse backbone.

A second obstacle lies in the supervision itself. Many prior methods implicitly collapse training toward a single deterministic solution, or encourage heatmaps that behave as if the combinatorial constraints should already be almost fully resolved by the neural model. Such supervision can increase the chance of directly decoding a feasible solution, but it can also over-concentrate the output and destroy its value as a search prior. In contrast, our target is induced by multiple high-quality solutions obtained in parallel. It does not treat every node uniformly: when several solutions agree, the target distribution becomes sharp; when several reasonable local structures coexist, the target remains diffuse. This uncertainty-aware supervision is precisely what makes the learned prior useful for downstream search rather than merely confident in isolation.

Core contributions of this paper Based on this positioning, the technical contributions of this paper can be summarized as follows:

- **A closed-loop LSL framework.** We formulate large-scale VRP solving as a bidirectional loop between learning and search, rather than a one-way model-plus-post-processing pipeline.
- **A globally-aware sparse backbone for ultra-large instances.** We design a lightweight sparse generative architecture that preserves global awareness without hard partitioning and makes the loop practical on consumer GPUs.

900 • **Multi-solution soft supervision from parallel search.** We convert multiple high-quality
901 search outcomes into a row-wise normalized soft target on the same sparse candidate graph,
902 allowing the model to learn sharp preferences where the structure is stable and diffuse priors
903 where search should remain active.

904 • **A practical self-improving system.** The same loop continuously reuses better search
905 results as stronger supervision, enabling scalable and effective self-refinement on large VRP
906 instances.

907 In short, the contribution of this paper is not a standalone neural solver and not a simple composition
908 of a model with a heuristic backend. The real contribution is a scalable closed-loop system in which
909 global sparse generation, search guidance, and uncertainty-aware supervision are designed to work
910 together.

911 **J Analysis of the computational complexity of LSL**

912 The main computational advantage of LSL comes from separating global neural prediction from
913 discrete search and from restricting the neural part to a sparse candidate graph.

914 **Sparse backbone complexity**

915 Let n be the number of rows in the row-slot representation, K the candidate width per row, d the
916 hidden dimension, and L the number of sparse attention layers. Unlike dense graph models that
917 must materialize all pairwise interactions, GSDDT only processes the $n \times K$ row-slot candidate
918 structure. The dominant attention, projection, and row-wise logit operations therefore scale with the
919 sparse support size rather than with n^2 . At a high level, the neural backbone has time complexity
920 proportional to $O(LnKd)$ and memory complexity proportional to $O(nK)$ up to constant factors
921 from multi-head projections and feed-forward layers.

922 **Why this differs from heavy constructive decoders**

923 In heavy constructive decoders, the neural model repeatedly performs sequence-level decision making
924 over a growing or changing partial solution, which can lead to cubic-in- n behavior across the full
925 construction process. LSL avoids this cost by predicting a sparse prior on the candidate graph in one
926 shot, without forcing the neural model to explicitly carry out the entire constructive trajectory.

927 **Heuristic refinement complexity**

928 The downstream search backend operates on the sparse candidate graph rather than on a dense $n \times n$
929 graph. Its exact runtime depends on the chosen local operators and the refinement budget I , but the
930 key point is that the neural part and the search part are both confined to a sparse support. This is what
931 makes the full learning–search loop practical on ultra-large instances and on consumer GPUs.

932 K Visualization

933 In this section, we provide graphical examples from both TSPLIB and CVRPLIB to illustrate
 934 representative search trajectories under learned sparse guidance. The figures should be read as
 935 intermediate refinement states rather than as a commitment to one specific constructive decoder. They
 936 highlight how local edits can gradually repair poor long edges and improve route structure while
 937 staying inside the candidate support preferred by the model.

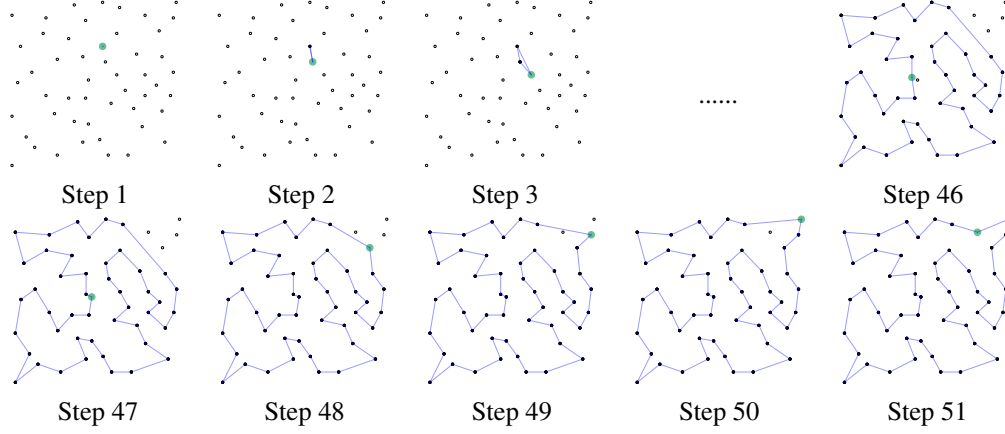


Figure 4: Representative refinement trajectory on the TSPLIB instance “eil51”. The green dot marks the node being edited at the current step. The hollow dots are unresolved nodes, and the black dots are nodes already incorporated into the current trajectory.

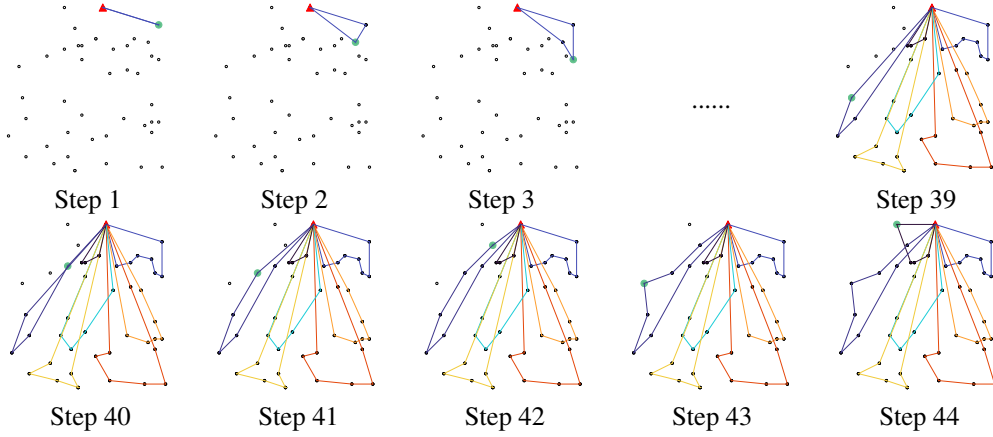


Figure 5: Representative refinement trajectory on the CVRPLIB instance “A-n45-k7”. The red triangle is the depot, the green dot marks the customer edited at the current step, the hollow dots are unresolved customers, and the black dots are customers already incorporated into the current route structure.

L Broader impacts

This research advances neural combinatorial optimization by introducing LSL, a closed-loop learning–search framework for large-scale Vehicle Routing Problems (VRPs). We hope this framework will provide useful insight for future work on scalable, search-aware neural optimization. As a general methodology for solving VRPs, LSL is not anticipated to create specific negative societal impacts beyond those already associated with optimization systems in logistics and routing.

M Licenses

The licenses for the codes and the datasets used in this work are listed in Table 13.

Table 13: List of licenses for the codes and datasets we used in this work

Resource	Type	Link	License
LKH3 [6]	Code	http://webhotel4.ruc.dk/~keld/research/LKH-3/	Available for academic research use
HGS [46]	Code	https://github.com/chkwon/PyHygese	MIT License
Concorde [45]	Code	https://github.com/jvkersch/pyconcorde	BSD 3-Clause License
POMO [12]	Code	https://github.com/yd-kwon/POMO	MIT License
Att-GCN+MCTS [30]	Code	https://github.com/SaneLYX/TSP_Att-GCRN-MCTS	MIT License
LEHD [13]	Code	https://github.com/CIAM-Group/NCO_code/tree/main/single_objective/LEHD	MIT License
BQ [14]	Code	https://github.com/naver/bq-nco	CC BY-NC-SA 4.0
GLOP [25]	Code	https://github.com/henry-yeh/GLOP	MIT License
HTSP [22]	Code	https://github.com/Learning4Optimization-HUST/H-TSP	MIT License
DIMES [31]	Code	https://github.com/DIMESTeam/DIMES	MIT License
DIFUSCO [32]	Code	https://github.com/Edward-Sun/DIFUSCO	MIT License
UDC [26]	Code	https://github.com/CIAM-Group/NCO_code/tree/main/single_objective/UDC-Large-scale-CO-master	MIT License
ELG [27]	Code	https://github.com/gaocrr/ELG	MIT License
INViT [28]	Code	https://github.com/Kasumigaoka-Utaha/INViT	Available for academic research use
Neuropt [36]	Code	https://github.com/yining043/Neuropt	MIT License
TSPLib [43]	Dataset	http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/	Available for any non-commercial use
CVRPLib [44]	Dataset	http://vrp.galgos.inf.puc-rio.br/index.php/en/	Available for academic research use

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction state the closed-loop LSL framework, its sparse scaling motivation, and its empirical scope, and these claims are matched by the method and experiment sections.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The paper includes a “Conclusion, Limitation, and Future Work” section that discusses current dependence on fixed heuristic operators, tighter backbone-search co-design, and broader problem coverage.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper is primarily algorithmic and empirical; it does not present theorem-proof style theoretical results that would require formal proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [No]

Justification: The paper describes the method, datasets, budgets, metrics, and some implementation details, but the submission does not yet disclose a complete reproduction package for all reported training and evaluation results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in

1052 some way (e.g., to registered users), but it should be possible for other researchers
1053 to have some path to reproducing or verifying the results.

1054 5. Open access to data and code

1055 Question: Does the paper provide open access to the data and code, with sufficient instruc-
1056 tions to faithfully reproduce the main experimental results, as described in supplemental
1057 material?

1058 Answer: [No]

1059 Justification: The manuscript cites public baselines and datasets, but this submission does not
1060 include an anonymized release of the authors' code, generated assets, or a full supplementary
1061 reproduction package.

1062 Guidelines:

- 1063 • The answer [N/A] means that paper does not include experiments requiring code.
- 1064 • Please see the NeurIPS code and data submission guidelines ([https://neurips.cc/
1065 public/guides/CodeSubmissionPolicy](https://neurips.cc/public/guides/CodeSubmissionPolicy)) for more details.
- 1066 • While we encourage the release of code and data, we understand that this might not
1067 be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not
1068 including code, unless this is central to the contribution (e.g., for a new open-source
1069 benchmark).
- 1070 • The instructions should contain the exact command and environment needed to run to
1071 reproduce the results. See the NeurIPS code and data submission guidelines ([https:
1072 //neurips.cc/public/guides/CodeSubmissionPolicy](https://neurips.cc/public/guides/CodeSubmissionPolicy)) for more details.
- 1073 • The authors should provide instructions on data access and preparation, including how
1074 to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- 1075 • The authors should provide scripts to reproduce all experimental results for the new
1076 proposed method and baselines. If only a subset of experiments are reproducible, they
1077 should state which ones are omitted from the script and why.
- 1078 • At submission time, to preserve anonymity, the authors should release anonymized
1079 versions (if applicable).
- 1080 • Providing as much information as possible in supplemental material (appended to the
1081 paper) is recommended, but including URLs to data and code is permitted.

1082 6. Experimental setting/details

1083 Question: Does the paper specify all the training and test details (e.g., data splits, hyperpa-
1084 rameters, how they were chosen, type of optimizer) necessary to understand the results?

1085 Answer: [No]

1086 Justification: The experimental sections and appendix specify datasets, evaluation protocols,
1087 and several inference settings, but they do not yet enumerate all training hyperparameters
1088 and optimization details needed for a full understanding of every result.

1089 Guidelines:

- 1090 • The answer [N/A] means that the paper does not include experiments.
- 1091 • The experimental setting should be presented in the core of the paper to a level of detail
1092 that is necessary to appreciate the results and make sense of them.
- 1093 • The full details can be provided either with the code, in appendix, or as supplemental
1094 material.

1095 7. Experiment statistical significance

1096 Question: Does the paper report error bars suitably and correctly defined or other appropriate
1097 information about the statistical significance of the experiments?

1098 Answer: [No]

1099 Justification: The reported experimental tables provide point estimates for cost, gap, and
1100 runtime, but the paper does not report error bars, confidence intervals, or formal significance
1101 tests.

1102 Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: The paper reports runtime numbers and notes consumer-GPU feasibility, but it does not provide a complete per-experiment hardware and memory specification for reproduction.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This is benchmark-based algorithmic research on routing optimization using synthetic and public problem instances, with no human-subject interaction or sensitive personal data, and the submission is anonymized.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

1155 Question: Does the paper discuss both potential positive societal impacts and negative
1156 societal impacts of the work performed?

1157 Answer: [No]

1158 Justification: The paper includes a short broader-impacts paragraph, but it does not yet
1159 provide a substantive discussion of concrete negative societal risks and mitigations beyond a
1160 brief statement that impacts are expected to be limited.

1161 Guidelines:

- 1162 • The answer [N/A] means that there is no societal impact of the work performed.
- 1163 • If the authors answer [N/A] or [No], they should explain why their work has no societal
1164 impact or why the paper does not address societal impact.
- 1165 • Examples of negative societal impacts include potential malicious or unintended uses
1166 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations
1167 (e.g., deployment of technologies that could make decisions that unfairly impact specific
1168 groups), privacy considerations, and security considerations.
- 1169 • The conference expects that many papers will be foundational research and not tied
1170 to particular applications, let alone deployments. However, if there is a direct path to
1171 any negative applications, the authors should point it out. For example, it is legitimate
1172 to point out that an improvement in the quality of generative models could be used to
1173 generate Deepfakes for disinformation. On the other hand, it is not needed to point out
1174 that a generic algorithm for optimizing neural networks could enable people to train
1175 models that generate Deepfakes faster.
- 1176 • The authors should consider possible harms that could arise when the technology is
1177 being used as intended and functioning correctly, harms that could arise when the
1178 technology is being used as intended but gives incorrect results, and harms following
1179 from (intentional or unintentional) misuse of the technology.
- 1180 • If there are negative societal impacts, the authors could also discuss possible mitigation
1181 strategies (e.g., gated release of models, providing defenses in addition to attacks,
1182 mechanisms for monitoring misuse, mechanisms to monitor how a system learns from
1183 feedback over time, improving the efficiency and accessibility of ML).

1184 11. Safeguards

1185 Question: Does the paper describe safeguards that have been put in place for responsible
1186 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
1187 image generators, or scraped datasets)?

1188 Answer: [N/A]

1189 Justification: The work studies routing optimization and does not describe release of high-
1190 risk generative models, scraped datasets, or similarly misuse-prone assets.

1191 Guidelines:

- 1192 • The answer [N/A] means that the paper poses no such risks.
- 1193 • Released models that have a high risk for misuse or dual-use should be released with
1194 necessary safeguards to allow for controlled use of the model, for example by requiring
1195 that users adhere to usage guidelines or restrictions to access the model or implementing
1196 safety filters.
- 1197 • Datasets that have been scraped from the Internet could pose safety risks. The authors
1198 should describe how they avoided releasing unsafe images.
- 1199 • We recognize that providing effective safeguards is challenging, and many papers do
1200 not require this, but we encourage authors to take this into account and make a best
1201 faith effort.

1202 12. Licenses for existing assets

1203 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
1204 the paper, properly credited and are the license and terms of use explicitly mentioned and
1205 properly respected?

1206 Answer: [Yes]

1207 Justification: The paper cites the baseline methods and benchmark datasets and includes a
1208 dedicated “Licenses” section with a table listing the asset sources and their reported licenses
1209 or usage terms.

1210 Guidelines:

- 1211 • The answer [N/A] means that the paper does not use existing assets.
- 1212 • The authors should cite the original paper that produced the code package or dataset.
- 1213 • The authors should state which version of the asset is used and, if possible, include a
1214 URL.
- 1215 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- 1216 • For scraped data from a particular source (e.g., website), the copyright and terms of
1217 service of that source should be provided.
- 1218 • If assets are released, the license, copyright information, and terms of use in the
1219 package should be provided. For popular datasets, `paperswithcode.com/datasets`
1220 has curated licenses for some datasets. Their licensing guide can help determine the
1221 license of a dataset.
- 1222 • For existing datasets that are re-packaged, both the original license and the license of
1223 the derived asset (if it has changed) should be provided.
- 1224 • If this information is not available online, the authors are encouraged to reach out to
1225 the asset’s creators.

1226 13. New assets

1227 Question: Are new assets introduced in the paper well documented and is the documentation
1228 provided alongside the assets?

1229 Answer: [No]

1230 Justification: The paper discusses bootstrapped large-scale training data and related outputs,
1231 but this submission does not include standalone documentation or an anonymized release
1232 package for newly introduced assets.

1233 Guidelines:

- 1234 • The answer [N/A] means that the paper does not release new assets.
- 1235 • Researchers should communicate the details of the dataset/code/model as part of their
1236 submissions via structured templates. This includes details about training, license,
1237 limitations, etc.
- 1238 • The paper should discuss whether and how consent was obtained from people whose
1239 asset is used.
- 1240 • At submission time, remember to anonymize your assets (if applicable). You can either
1241 create an anonymized URL or include an anonymized zip file.

1242 14. Crowdsourcing and research with human subjects

1243 Question: For crowdsourcing experiments and research with human subjects, does the paper
1244 include the full text of instructions given to participants and screenshots, if applicable, as
1245 well as details about compensation (if any)?

1246 Answer: [N/A]

1247 Justification: The paper does not involve crowdsourcing, user studies, or any other research
1248 with human subjects.

1249 Guidelines:

- 1250 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
1251 with human subjects.
- 1252 • Including this information in the supplemental material is fine, but if the main contribu-
1253 tion of the paper involves human subjects, then as much detail as possible should be
1254 included in the main paper.
- 1255 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,
1256 or other labor should be paid at least the minimum wage in the country of the data
1257 collector.

1258	15. Institutional review board (IRB) approvals or equivalent for research with human subjects
1259	
1260	Question: Does the paper describe potential risks incurred by study participants, whether
1261	such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
1262	approvals (or an equivalent approval/review based on the requirements of your country or
1263	institution) were obtained?
1264	Answer: [N/A]
1265	Justification: No human participants are involved in this work, so IRB review and participant-
1266	risk disclosure are not applicable.
1267	Guidelines:
1268	• The answer [N/A] means that the paper does not involve crowdsourcing nor research
1269	with human subjects.
1270	• Depending on the country in which research is conducted, IRB approval (or equivalent)
1271	may be required for any human subjects research. If you obtained IRB approval, you
1272	should clearly state this in the paper.
1273	• We recognize that the procedures for this may vary significantly between institutions
1274	and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
1275	guidelines for their institution.
1276	• For initial submissions, do not include any information that would break anonymity (if
1277	applicable), such as the institution conducting the review.
1278	16. Declaration of LLM usage
1279	Question: Does the paper describe the usage of LLMs if it is an important, original, or
1280	non-standard component of the core methods in this research? Note that if the LLM is used
1281	only for writing, editing, or formatting purposes and does <i>not</i> impact the core methodology,
1282	scientific rigor, or originality of the research, declaration is not required.
1283	Answer: [N/A]
1284	Justification: LLMs are not part of the proposed method, experiments, or scientific claims in
1285	the paper.
1286	Guidelines:
1287	• The answer [N/A] means that the core method development in this research does not
1288	involve LLMs as any important, original, or non-standard components.
1289	• Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1290	be described.